

网络安全协议

顾辰
合肥工业大学

guchen@hfut.edu.cn



Web安全需求

- 当今所有企业、大部分政府机构和很多个人都拥有自己的网站。个人和公司访问互联网的数量正在急剧增加。但是，实际情况是互联网或Web都极易受到各种各样的泄密攻击。
- 万维网(World Wide Web, WWW)从根本上说是一个运行于互联网和TCP/IP上的客户端/服务器应用系统。Web提出了一系列与一般计算机和网络安全不太相同的新挑战。
- 互联网是双向的。与传统的发布环境不同，电子发布系统将涉及电子文本、电视广播、语音应答或传真反馈等，使得Web服务器容易受到来自互联网的攻击。
- Web服务器相对容易配置和管理、Web内容也越来越容易开发，但是其底层软件却极其复杂。这种复杂的软件中可能会隐藏很多潜在的安全缺陷。

Web安全需求

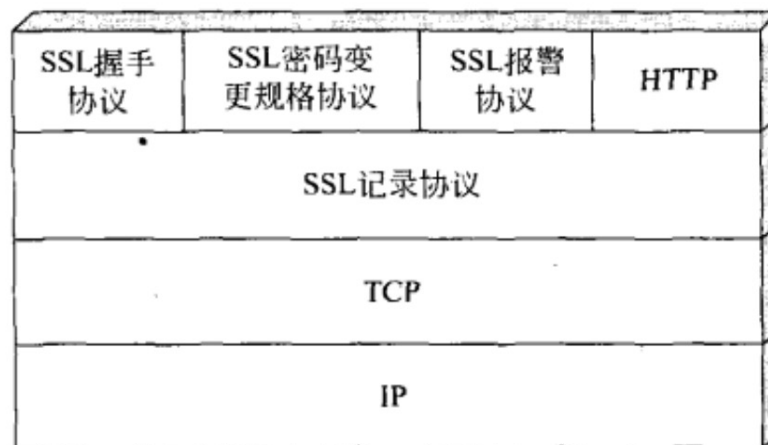
- 安全威胁可以分为主动攻击和被动攻击。被动攻击包括在浏览器和服务器之间通信时窃听和获取Web站点上受限访问信息的访问权。主动攻击包括用户假冒、在客户和服务端之间的传输过程中替换消息和更改Web站点上的信息等。

- 按照威胁发生的地域分为三类：Web服务器、浏览器、浏览器与服务端之间的网络通信。通信的安全问题可归结为网络安全。

类 型	威 胁	后 果	对 策
完整性	用户数据修改 特洛伊木马浏览器 内存修改 更改传输中的消息	丢失消息 设备受损 易受其他威胁	密码校验和
机密性	网上窃听 窃取服务器信息 窃取客户端信息 窃取网络配置信息 窃取客户与服务端通话信息	信息丢失 秘密丢失	加密 Web 委托代理
拒绝服务	破坏用户线程 用假消息使机器溢出 填满硬盘或内存 用 DNS 攻击孤立机器	中断 干扰 阻止用户完成任务	难以防止
认证	假冒合法用户 伪造数据	用户错误 相信虚假信息	密码技术

TLS

- ❑ 传输层安全(TLS)是广泛使用的一种安全服务,其当前版本更新到1.2。TLS是一种Internet标准,它是从称为安全套接字层(SSL)的商业协议发展而来。
- ❑ TLS是一种通用服务,是依赖TCP实现的一组协议。大多数浏览器都配备了TLS,大多数Web服务器都实现了该协议。
- ❑ TLS使用TCP提供一种可靠的端对端的安全服务。TLS不是单个协议,它由两层协议组成。
- ❑ TLS记录协议对各种更高层协议提供基本的安全服务。尤其是,超文本传输协议(Hypertext Transfer Protocol, HTTP)是为Web客户端/服务器的交互提供传输服务的协议,它可以在TLS的顶层运行。
- ❑ TLS中定义的三个较高层协议分别是:握手协议、修改密码规格协议和报警协议。这些TLS协议规范用来管理TLS的交换。



TLS

- TLS协议中的两个重要概念是TLS会话和TLS连接。
 - 连接：连接是一种能够提供合适服务类型(按照OSI分层模型定义)的传输。对TLS来说，这种连接是点对点的关系而且都是短暂的。每一条连接都与一个会话相关联。
 - 会话：TLS会话是客户与服务器之间的一种关联。会话是通过握手协议来创建的。所有会话都定义了密码安全参数集合，这些参数可以在多个安全连接之间共享。会话通常用来减少每次连接建立安全参数的昂贵协商费用。
- 实际上还有若干个状态与每个会话相关联。一旦建立起一个会话，对于读和写(即接收和发送)就存在一个当前操作状态。此外，在握手协议中，还会创建读挂起和写挂起状态。当握手协议结束后，挂起的状态又回到当前状态。

TLS

□ 会话状态由下列参数定义：

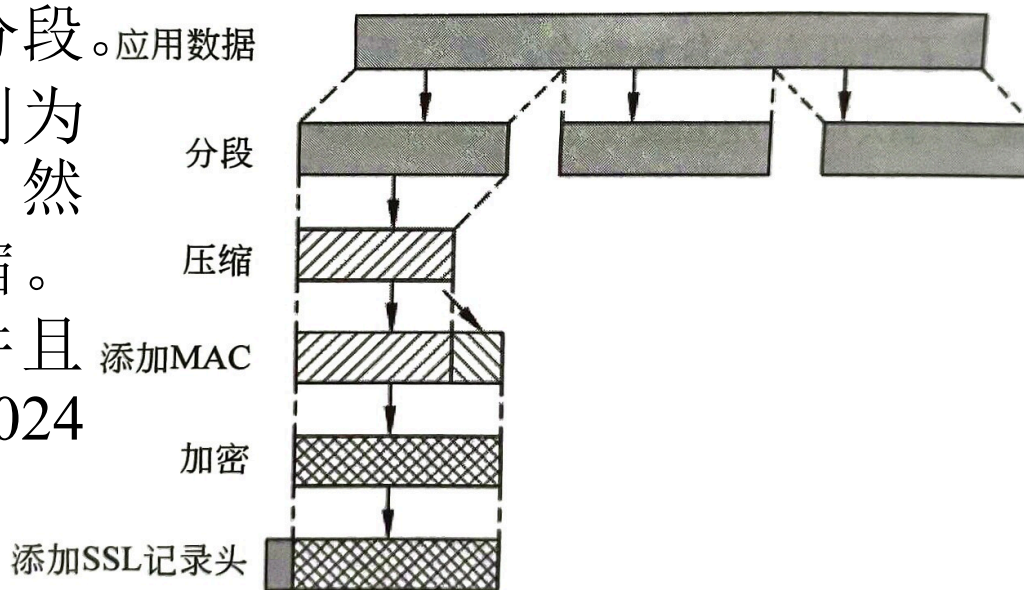
- 会话标识符：由服务器产生的用于标识活动或可恢复的会话状态的一个任意字节序列。
- 对等实体证书：对等实体的X.509证书。
- 压缩方法：加密前用于压缩数据的算法。
- 密码规格：包括数据加密算法(例如，AES算法等)规格和用于计算MAC的散列算法(如MD5或SHA-1算法等)规格等。
- 主密钥：客户端和服务端共享的48字节的会话密钥。
- 可恢复性：表明会话是否可被用于初始化新连接的标志。

□ 连接状态由下列参数定义：

- 服务器和客户端随机数：由服务器和客户端为每个连接选定的字节串。服务器写MAC密钥：服务器发送数据时用于计算MAC值的密钥。
- 客户端写MAC密钥：客户端发送数据时用于计算MAC值的密钥。
- 服务器写密钥：服务器用于加密数据、客户端用于解密数据的加密密钥。客户端写密钥：客户端用于加密数据、服务器用于解密数据的对称加密密钥。
- 初始化向量：在CBC模式中，需要为每个密钥配置一个初始化向量(IV)，最初的IV值由TLS的握手协议初始化。之后，每个记录的最后一个密码块被保存，作为后续记录的IV。
- 序列号：建立连接的各方为每条连接发送和接收的消息维护单独的序列号。

TLS记录协议

- TLS记录协议为TLS连接提供如下两种服务。
 - 机密性：定义一个可以用于加密TLS载荷的传统加密共享密钥。
 - 消息完整性：定义一个用于产生消息认证码(MAC)的共享密钥。
- TLS运行的第一步是分段。应用数据把每个上层消息分割为不大于24字节的块。然后选择压缩或不压缩。压缩必须是无损的并且增加长度不能超过1024字节。

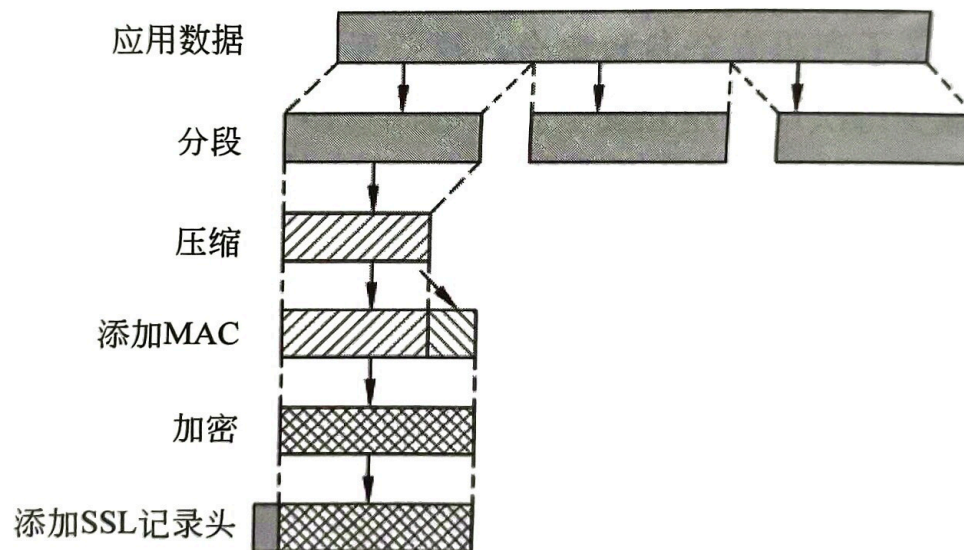


TLS记录协议

- 接下来的处理步骤是在压缩数据的基础上计算消息认证码。TLS使用RFC2104中定义的HMAC算法。可知HMAC定义为：

$$\text{HMAC}(K, M) = H[(K' \oplus \text{opad}) || H[(K' \oplus \text{ipad}) || M]]$$
。其中：

- H=嵌入的哈希函数(对TLS，或者MD5或者SHA-1)
- M=输入到HMAC的信息
- K'=左边填充零后的密钥，填充零的目的是为了使结果与哈希码的块长度相同（对MD5和SHA-1，块长度=512比特）
- ipad=00110110(36的十六进制)重复64次(512比特)
- opad=01011100(5C的十六进制)重复64次(512比特)



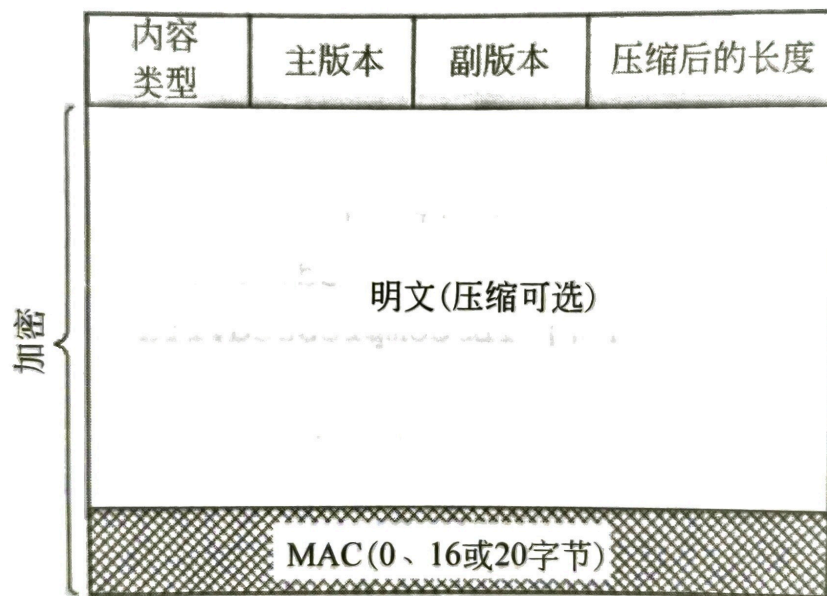
TLS记录协议

- 对流密码算法，压缩后的消息与MAC码一起被直接加密。值得注意的是MAC码是在加密前计算得到的。计算所得的MAC码连同明文或压缩后的明文一同被加密。
- 对分组密码，可以在加密之前在MAC之后添加填充。填充是以多个填充字节的形式，后跟填充长度的单字节指示。填充可以是总数中的任何数量的结果，其长度是密码的块长度的倍数，最大可到255字节。例如，如果密码的块长度是16字节(例如AES)或者如果某明文(或压缩后的明文)加上MAC再加上填充的长度是79字节，填充长度可以是1、17、33等，最大为161字节。如果填充长度为161字节，总长度为79字节+161字节=240字节。可变填充长度可用于阻止基于分析交换消息长度的攻击。

分组密码		流密码	
算 法	密钥大小	算 法	密钥大小
AES	128 或 256	RC4-128	128
3DES	168		

TLS记录协议

- ❑ TLS记录协议处理过程的最后一步是添加一个由下列域组成的TLS头：
 - 内容类型(8比特): 用于处理封装分段的高层协议。
 - 主版本号(8比特): 表明应用的TLS协议的主版本号。对于TLSv3.0, 该值为3。
 - 副版本号(8比特): 表明应用的TLS协议的从版本号。对于TLSv3.0, 该值为0。
 - 压缩后的长度(8比特): 以字节为单位的明文块(如果使用了压缩, 则为压缩后的明文块)长度。
- ❑ 内容类型被定义为修改密码规格、报警、握手和应用数据4种, 前3种是对TLS特定的协议。值得注意的是, 在各种应用(例如, HTTP)中使用TLS没有什么限制, 它们提供的数据内容对TLS来说是不透明的。



修改密码规格协议

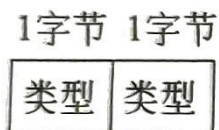
- 修改密码规格协议是应用TLS记录的4个TLS规格协议之一，也是最简单的一个协议。本协议包含一条信息，由1个值为1的字节组成。
- 这条消息的唯一功能是使得挂起状态改变为当前状态，用于更新此连接使用的密码套件。



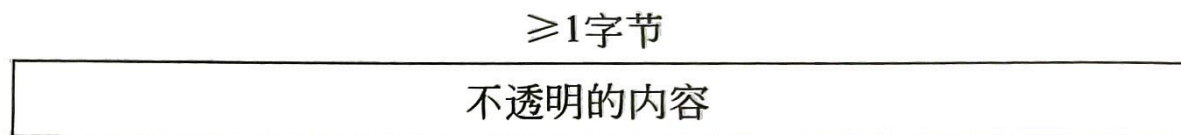
(a) 修改密码规格协议



(c) 握手协议



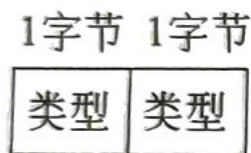
(b) 报警协议



(d) 其他上层协议(如HTTP)

报警协议

- 警报协议用于将与TLS相关的警报传达给对等实体。与使用TLS的其他应用一样，警报消息也要按照当前状态的规格进行压缩和加密操作。
- 这一协议过程中的每一条消息都由两个字节组成。其中第一个字节可以取值为警告(1)或致命(2)以表示消息的严重程度。如果严重程度为致命，TLS将立即结束当前连接。虽然该会话中的其他连接还可以继续进行，但是本次会话不允许建立新的连接。第二个字节包括一种用于指明具体警告的编码。下面列出致命警告的内容(由TLS规范定义):
 - 非预期消息：接收到不恰当的消息。
 - MAC记录出错：接收到不正确的MAC码。
 - 解压缩失败：解压缩函数接收到不恰当的输入。
 - 握手失败：发送方不能协商出一组可接受的安全参数。
 - 不合法参数：握手消息中的域超出范围或与其他域不一致。
 - 解密失败：以无效方式解密的密文，或者它不是块长度的偶数倍或其填充值，如果选中，则不正确。



(b) 报警协议

报警协议

- 记录溢出：收到的TLS，其长度超过有效载荷(密文)。
- 未知CA：收到了有效的证书链或部分链，但未接受证书，因为无法找到CA-证书或无法与已知的可信CA匹配。
- 拒绝访问：收到有效证书，但在应用访问控制时，发送方决定不继续进行协商。
- 解码错误：无法解码消息，因为字段超出其指定范围或消息长度不正确。
- 出口限制：检测到不符合关键长度出口限制的谈判。
- 协议版本：客户端尝试协商的协议版本已被识别但不受支持。
- 安全性不足：当协商失败时，返回一个不是握手失败的信息，因为服务器要求的密码比客户端支持的密码更安全。
- 内部错误：与对等方无关的内部错误或协议的正确性使其无法继续。
- 结束通知：通报接收方，发送方在本次连接上将不再发送任何消息。连接双方中的一方在关闭连接之前都应该给对方发送这样一条消息。

报警协议

- 没有证书：如果没有合适的证书可用时，发送这条消息作为对证书请求者的回应。
- 证书不可用：接收到的证书不可用(例如包含的签名无法通过验证)。
- 不支持的证书：不支持接收到的证书类型。
- 证书作废：证书已被签发者吊销。
- 证书过期：证书已过期。
- 未知证书：处理证书过程中引起的其他未知问题，导致该证书无法被系统识别和接受。
- 解密错误：握手加密操作失败，包括无法验证签名，解密密钥交换，验证已完成的消息。
- 用户取消：由于与协议故障无关的某些原因，此握手被取消。
- 不重新谈判：由客户端响应“hello”请求，或服务器在初始握手后响应客户端“hello”而发送。这些消息中的任何一个通常都会导致重新谈判，但此警报表示发送方无法重新谈判。此消息始终是一个警告。

握手协议

- TLS最复杂的部分是握手协议。这一协议允许客户端和服务端相互认证，并协商加密和MAC算法，以及用于保护TLS记录中所发送数据加密密钥。握手协议在任何应用数据被传输之前使用。
- 握手协议由客户端和服务端之间的一系列消息交换组成。每一条消息都包括三个域：
- 类型(1字节):表示预定义的10种消息类型之一。
- 长度(3字节):以字节为单位的消息长度。
- 内容(≥ 0 字节):和本条消息相关的参数。

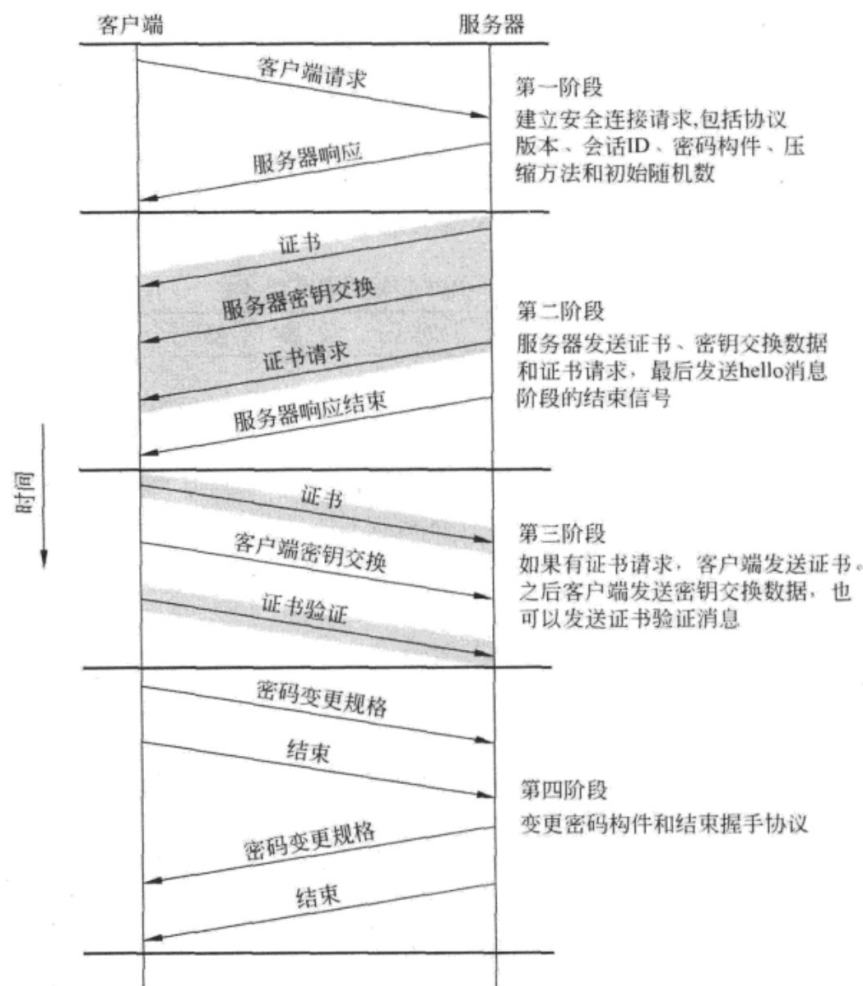
1字节	3字节	≥ 0 字节
类型	长度	内容

(c) 握手协议

消息类型	参 数
请求	空
客户端请求	版本, 随机数, 会话 ID, 密码套件, 压缩方法
服务器响应	版本, 随机数, 会话 ID, 密码套件, 压缩方法
证书	X.509v3 证书链
服务器密码交换	参数, 签名
证书请求	类型, 授权
服务器响应结束	空
证书验证	签名
客户端密钥交换	参数, 签名
结束	散列值

握手协议

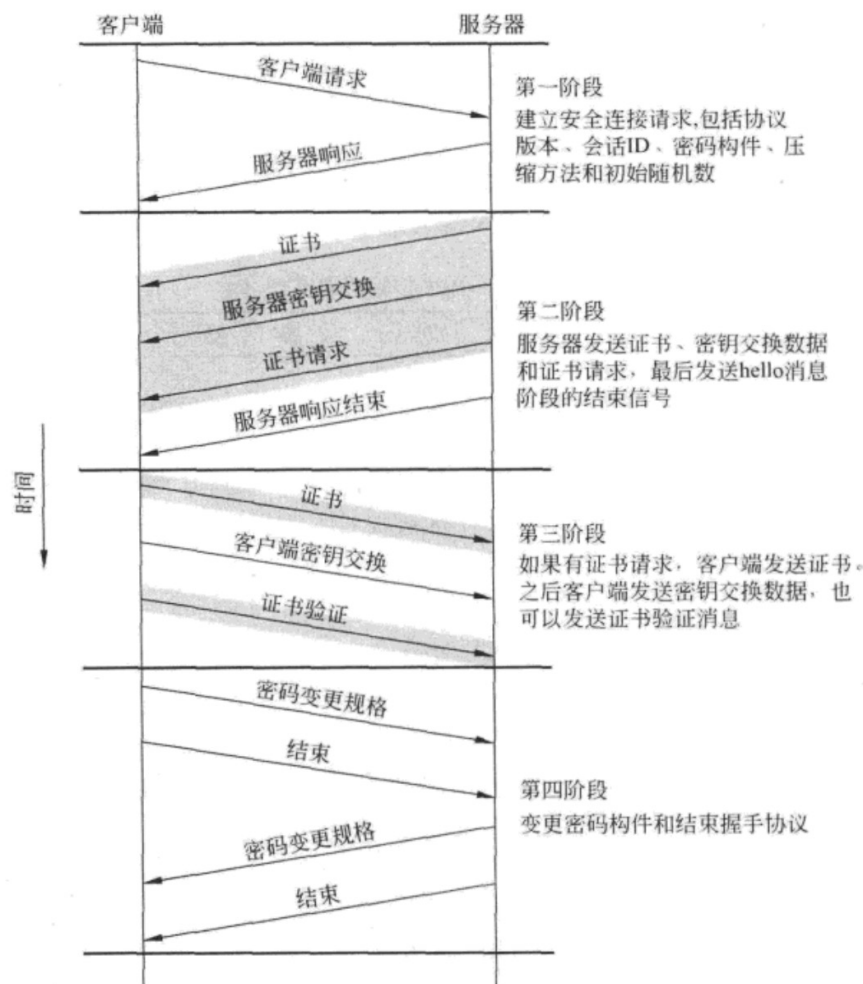
- 为建立客户端和服务端之间的逻辑连接需要进行的初始交换。这些交换可分为4个阶段。
- **第一阶段:客户端发起建立连接请求。**这一阶段主要是发起逻辑连接并建立与之关联的安全能力。交换首先由客户端通过发送下列 `client_hello` 消息启动。
 - 版本:客户端的SSL最高版本。
 - 随机数:由客户端产生的随机序列,由32比特时间戳以及安全随机数生成器产生的28字节随机数组成。这些数主要用于密钥交换过程中防止重放攻击。



注:加阴影的传输是可选的,或者是与情况相关的消息,它们并非总会被发送

握手协议

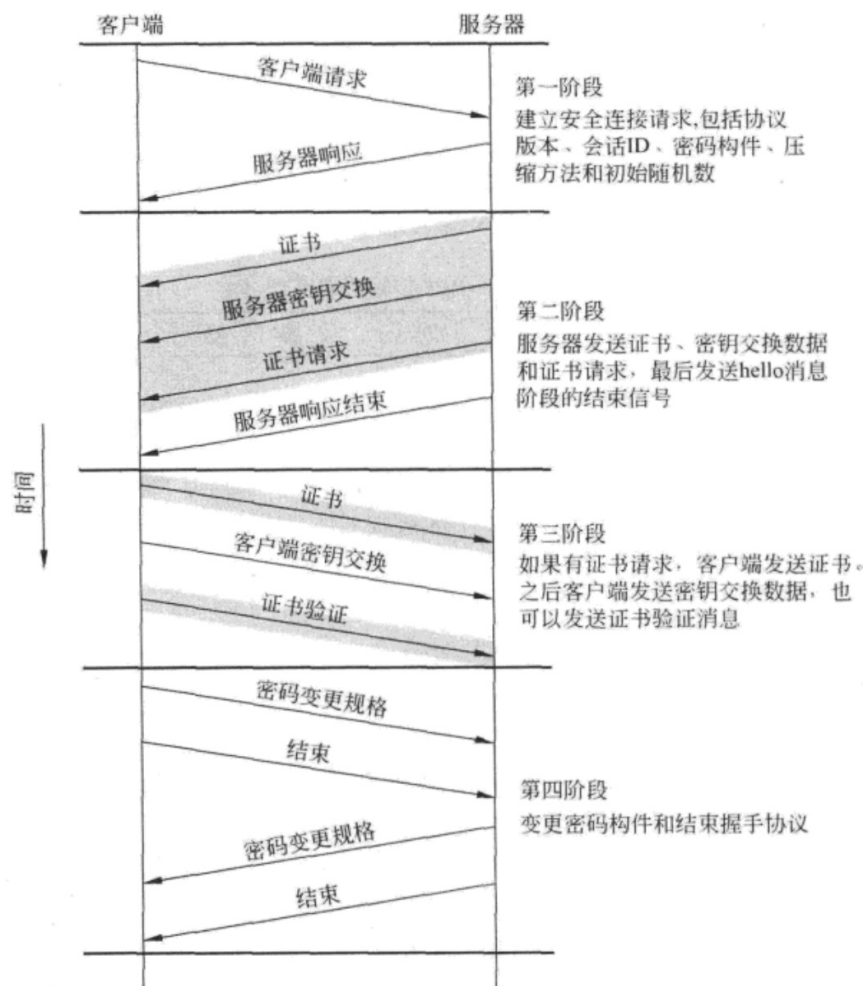
- 会话ID: 可变长度的会话标识符。非零值表示客户端希望更新现有连接的参数。零值表示客户端希望为这次会话建立一条新连接。
- 密码套件: 该列表包括客户端支持的密码算法组合，以优先选用递减的顺序给出。列表中的每一行(即每一个密码构件)同时定义密钥算法和密码规格。
- 压缩方法: 客户端支持的压缩方法列表。



注: 加阴影的传输是可选的, 或者是与情况相关的消息, 它们并非总会被发送

握手协议

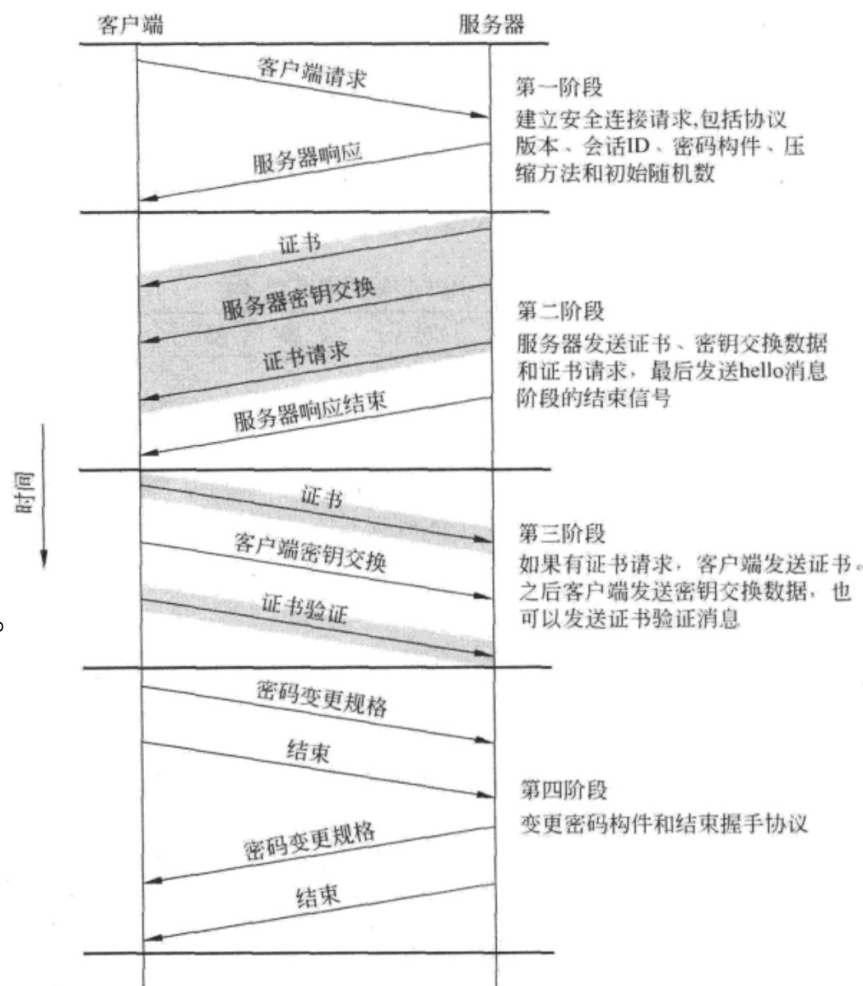
- 发送完client_hello消息后，客户端将等待server_hello消息，该消息所包含的参数与client_hello消息包含的参数相同。同时server_hello消息遵循以下的惯例。版本域包含客户端建议的较低版本和服务器建议的最高版本。
- 服务器产生一个独立于客户端随机域的新随机数域。如果客户端会话ID域的值非零，那么服务器应采用相同的取值。否则，服务器的会话ID域将包括一个新会话值。



注:加阴影的传输是可选的,或者是与情况相关的消息,它们并非总会被发送

握手协议

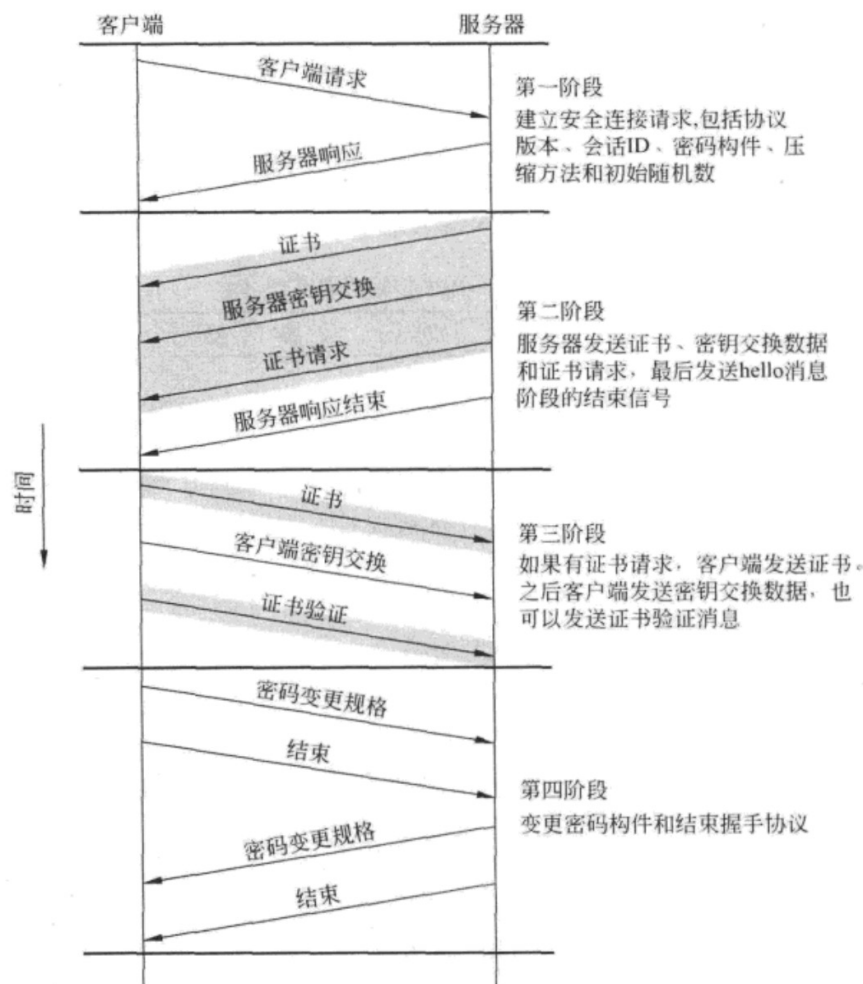
- ❑ 服务器从客户端提供的可选方案中选定的唯一一组密码套件。压缩域包括服务器从客户端建议中选定的压缩方法。
- ❑ 密码套件参数的第一项内容是密钥交换方法(建立传统加密的密钥和MAC密钥的交换)。协议支持下列密钥交换方法。
 - RSA:密钥用接收者的RSA公钥加密。这里必须要有一个接收者的可用公钥证书。



注:加阴影的传输是可选的,或者是与情况相关的消息,它们并非总会被发送

握手协议

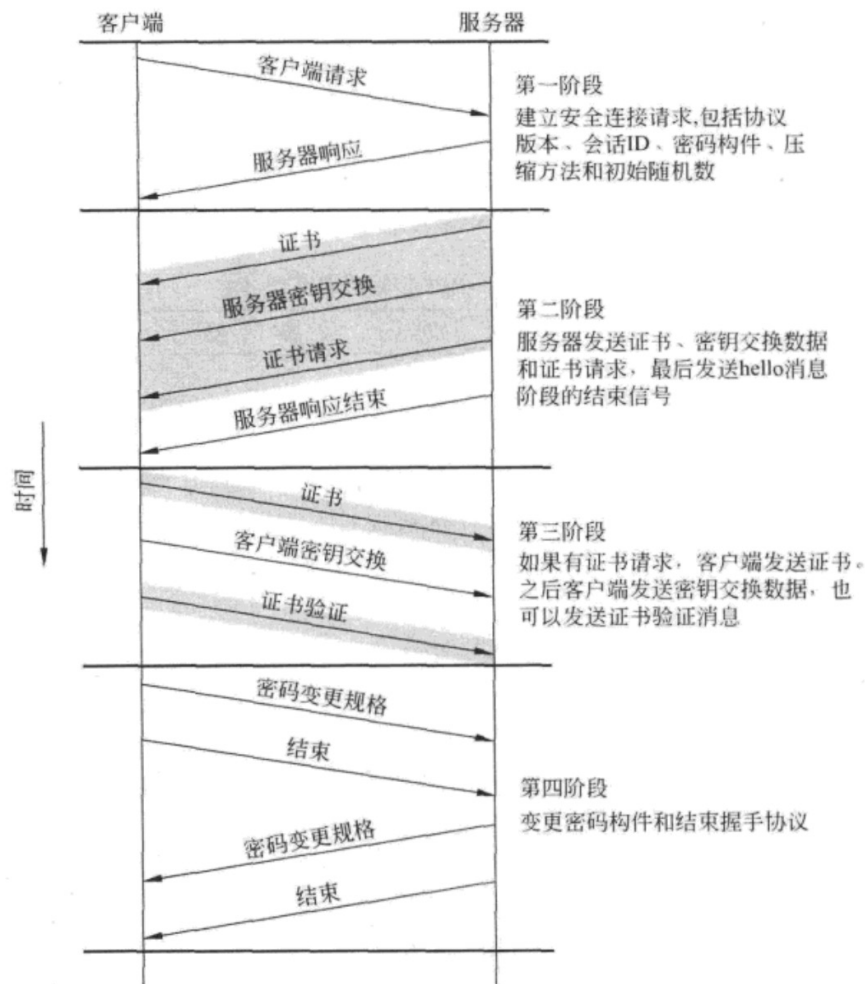
- 紧随密钥交换方法约定之后的是密码规格。
- 密码算法:可以是前面提到的算法中的任何一种:RC4、RC2、DES、3DES、DES40和IDEA。
- MAC算法: SHA-1等。
- 密码类型:流密码或分组密码。
- 散列长度:0、16(用于MDS)或20(用于SHA-1)字节。
- 密钥材料:一个字节串(其中包含用于产生写密钥的数据)。
- IV大小: 密码分组连接(CBC)加密模式中初始向量的大小。



注:加阴影的传输是可选的,或者是与情况相关的消息,它们并非总会被发送

握手协议

- 第二阶段:服务器认证和密钥交换。发送的消息包括一个X.509证书。证书消息对于密钥交换方法是必需的。



注:加阴影的传输是可选的,或者是与情况相关的消息,它们并非总会被发送

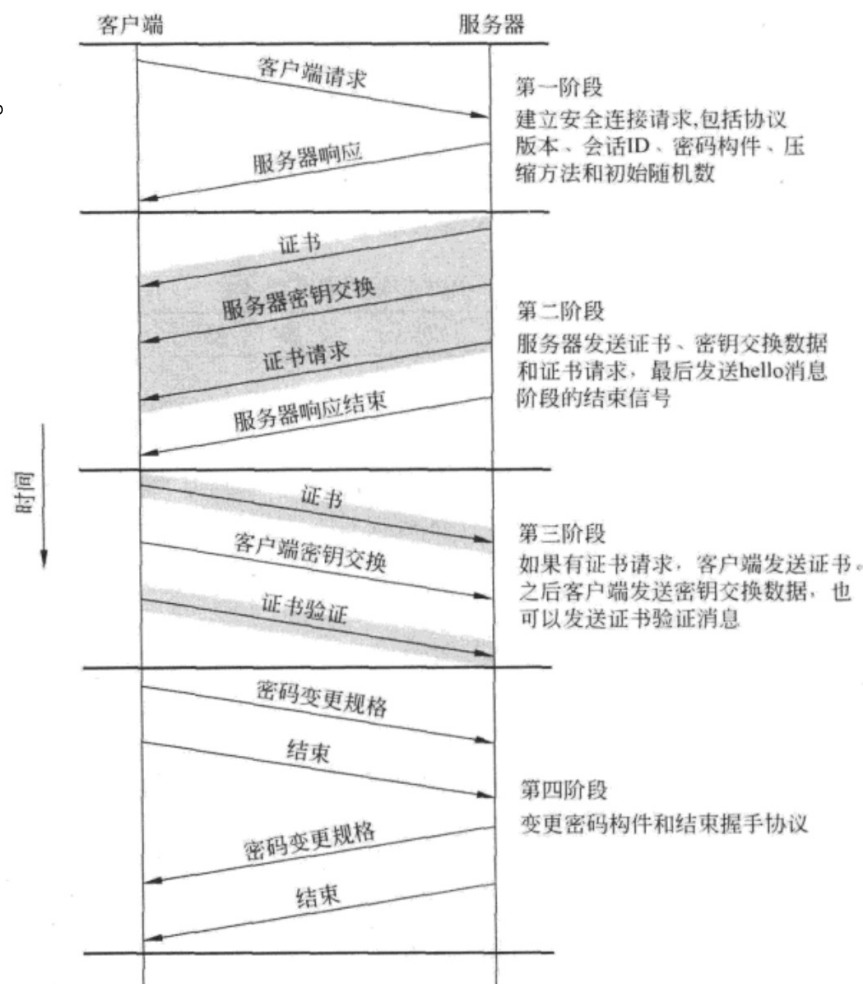
握手协议

➤ 如果有必要，将发送一个服务器密钥交换（`server_key_exchange`）消息。在下列两种情况下无需发送该消息：(1) 服务器已发送包含固定Diffie-Hellman参数的证书；(2) 使用了RSA密钥交换方法。

➤ 得到消息的散列值之后使用发送者的私钥对它加密就可得到签名。散列定义如下：

$\text{Hash}(\text{ClientHello.random} \parallel \text{ServerHello.random} \parallel \text{ServerParams})$

第二阶段中的最后一条消息是 `server_done` (服务器结束) 消息。该消息由服务器发出并示意服务器的hello及相关消息已经结束。该消息没有参数，发送完这个消息后，服务器要等待客户的响应。

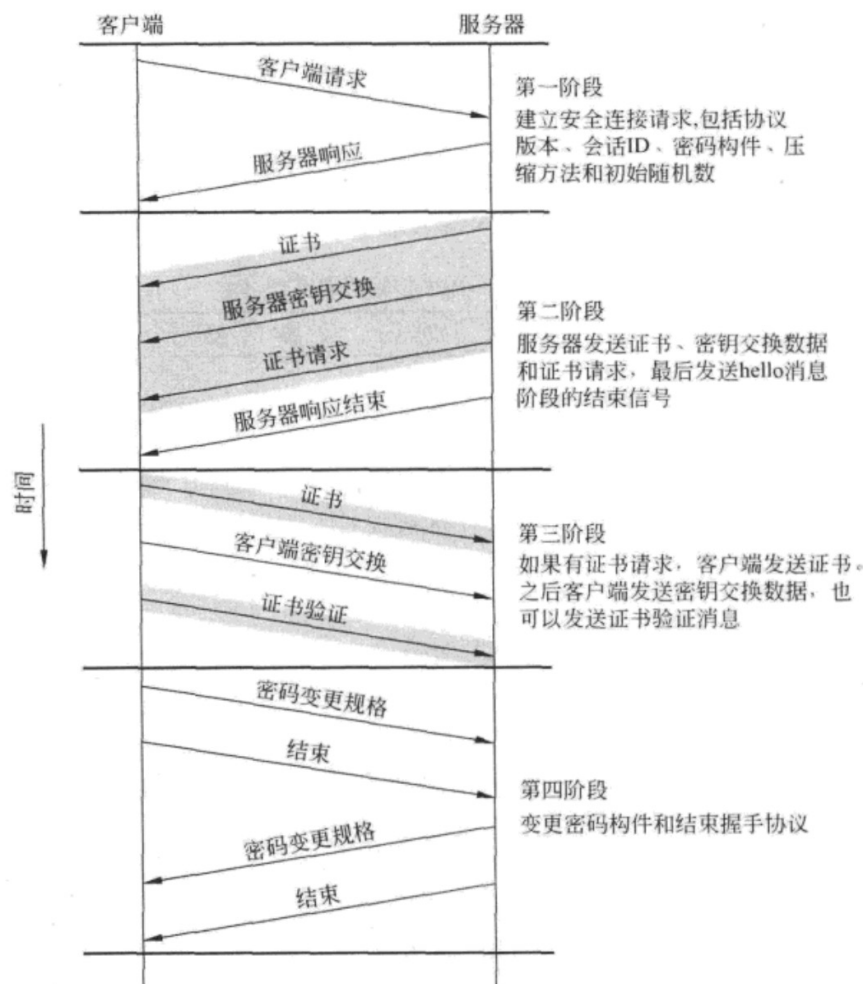


注：加阴影的传输是可选的，或者是与情况相关的消息，它们并非总会被发送

握手协议

- 第三阶段:客户端认证和密钥交换。接收到server_done(服务器结束)消息后, 如果需要, 客户端应该验证服务器提供的证书是否有效, 同时还要检查server_hello参数是否是可接受的。如果所有这些条件均满足, 那么客户端将返回一条或更多消息给服务器。

➤ 如果服务器已请求证书, 则以客户端发送一条证书消息为这一阶段的开始。如果没有合适的证书可用, 那么客户端发送一个no_certificate alert(无证书警报)。



注: 加阴影的传输是可选的, 或者是与情况相关的消息, 它们并非总会被发送

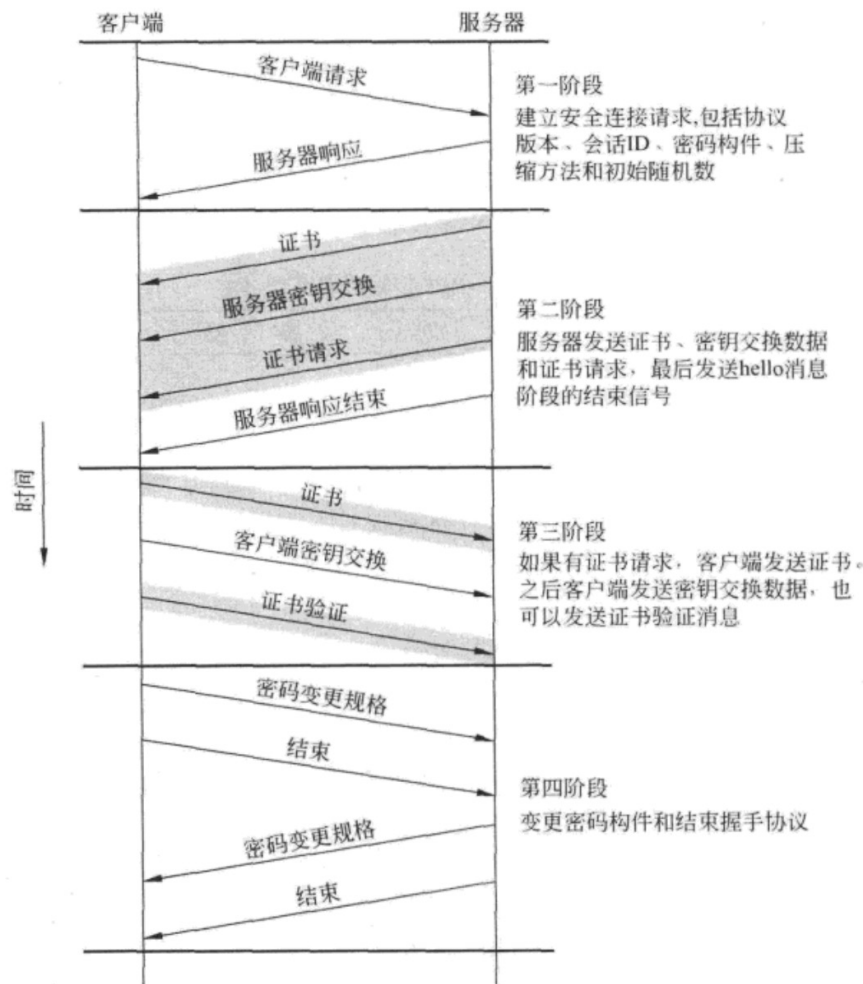
握手协议

➤ 接下来是client_key_exchange(客户端密钥交换)消息，该消息必须在这一阶段发送。消息内容由下列的密钥交换类型决定：

➤ RSA:客户端产生一个48字节的pre-master_secret(预备主密钥)，并用从服务器证书中得到的公钥或者用从server_key_exchange消息中得到的RSA临时密钥进行加密。

➤ 暂态或匿名Diffie-Hellman:发送客户端的Diffie-Hellman公钥参数。

➤ 固定Diffie-Hellman:以证书消息的形式发送客户端的Diffie-Hellman公钥参数，该消息内容为空。



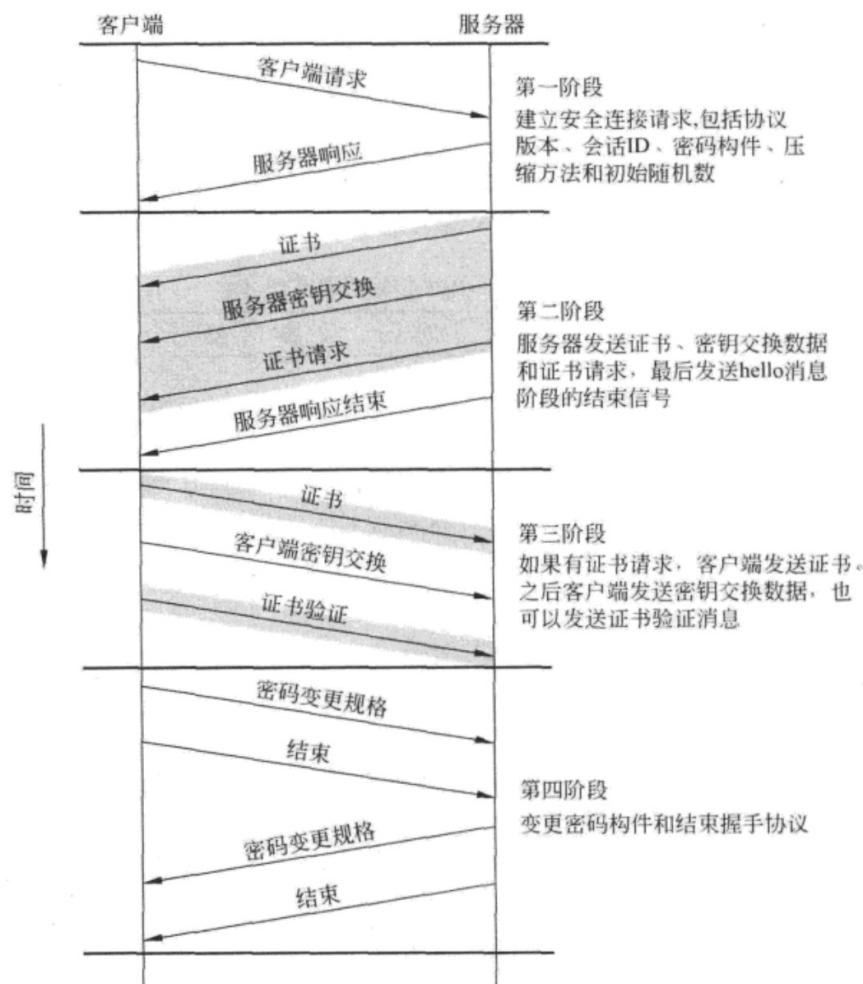
注:加阴影的传输是可选的,或者是与情况相关的消息,它们并非总会被发送

握手协议

➤ 最后在这一阶段，客户端可以发送 `certificate_verify`(证书验证)消息，以便对客户端证书进行显式验证。仅当客户端证书具有签名功能(也就是说，除包含固定Diffie-Hellman参数的所有证书)时才会发送该消息。这个消息是对一个散列码的签名，该散列码基于前面的消息，定义如下：

➤ `CertificateVerify.signature.md5_hash`
`MD5(handshake_messages);`

其中，`handshake_message`（握手消息）是客户端启动`client_hello`时发送或接收到的所有握手协议消息，但不包括`client_hello`消息本身。

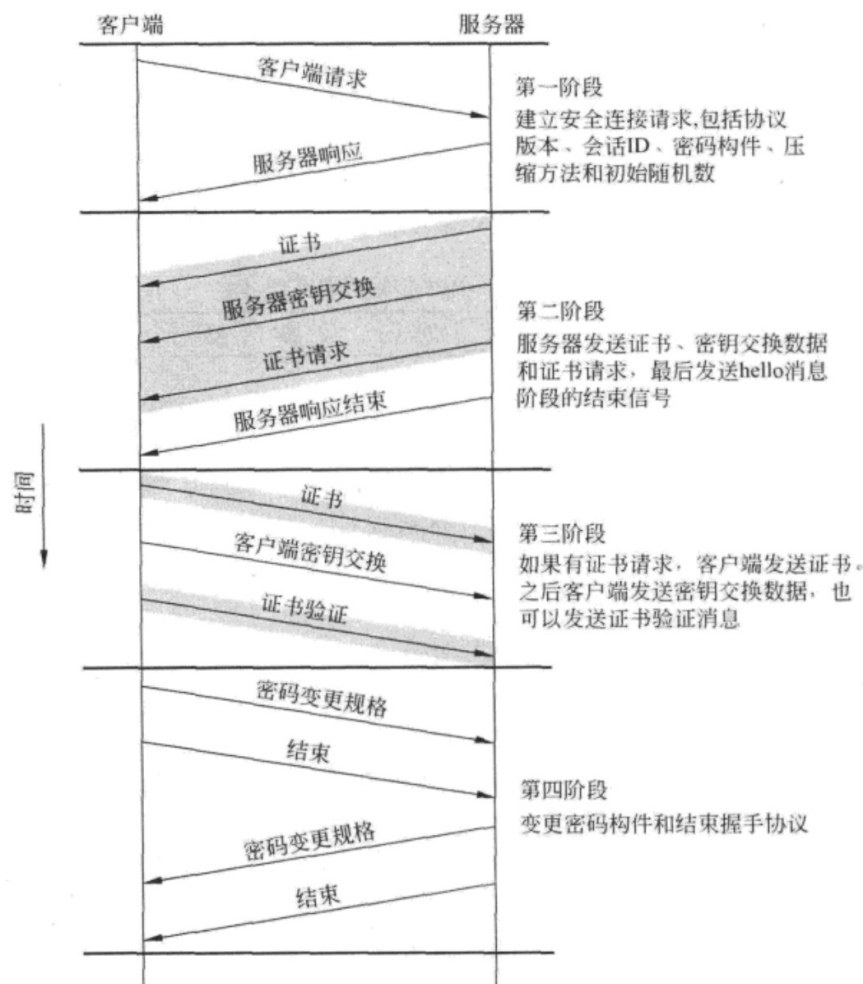


注:加阴影的传输是可选的,或者是与情况相关的消息,它们并非总会被发送

握手协议

□ 第四阶段:完成。这一阶段完成安全连接的建立。客户端发送一个 `change_cipher_spec`(密码变更规格) 消息, 并把挂起的密码规格复制到当前密码规格中。

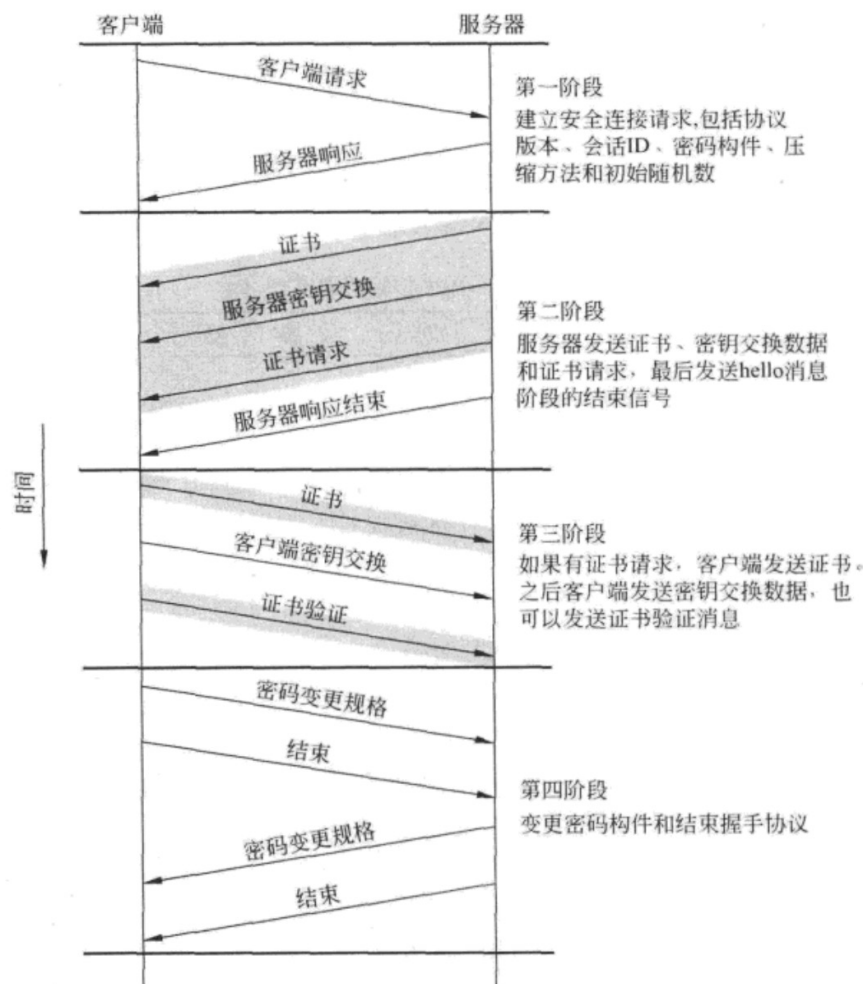
➤ 该消息不是握手协议的一部分, 而是使用密码变更规格协议发送的。客户端在新算法、新密钥和新秘密值下立即发送 `finished`(结束) 消息。 `finished` 消息用于验证密钥交换和认证过程是否成功。



注: 加阴影的传输是可选的, 或者是与情况相关的消息, 它们并非总会被发送

握手协议

- finished消息是两个散列码的串接:
 $\text{PRF}(\text{master_secret}, \text{finished_label}, \text{MD5}(\text{handshake_messages}) \parallel \text{SHA-1}(\text{handshake_messages}))$
- 其中, 结束标签(finished_label)对客户端来说是“client finished”字符串, 对服务器来说是“server finished”字符串。
- 作为对客户发送的这两条消息的响应, 服务器发送自己的change_cipher_spec_message(密码变更规格消息), 把未定的密码规格转变为当前的密码规格并发送其finished消息。到此为止握手过程已经完成, 客户端与服务器可以开始交换应用层数据。



注: 加阴影的传输是可选的, 或者是与情况相关的消息, 它们并非总会被发送

握手协议

- ❑ 在握手协议中的密码计算部分，目前存在的两个问题：
 - 如何通过密钥交换创建一个共享主密钥？
 - 如何从共享主密钥中产生密码参数？
- ❑ **主密钥的创建。**共享主密钥是通过安全密钥交换方式为本次会话创建的一个一次性48字节(384比特)的值。创建过程分两步完成。第一步:交换预备主密钥。第二步:双方计算主密钥。对预备主密钥的交换，有下面两种情况：
 - **RSA:**客户端产生一个48字节的预备主密钥，并使用服务器的RSA公钥加密，然后将其发送给服务器。服务器使用自己的私钥解密以得到pre_master_secret(预备主密钥)。
 - **Diffie-Hellman:**服务器和客户端各自产生一个Diffie-Hellman公钥值。交换之后，双方再分别做Diffie-Hellman计算来创建共享的预备主密钥。

握手协议-密码计算

- 现在，客户和服务端都按照下面方法计算主密钥：

$$\text{Master_secret} = \text{PRF}(\text{pre_master_secret}, \text{"master secret"}, \\ \text{ClientHello.random} \parallel \text{ServerHello.random})$$

- 其中，ClientHello.random和ServerHello.random是在初始hello消息中交换的两个随机数。这个算法一直要执行到48字节的伪随机输出都产生完为止。

- 密钥块的计算(MAC密钥、会话加密密钥和IV)定义如下：

$$\text{Key_block} = \text{PRF}(\text{SecurityParameters.master_secret}, \text{"key expansion"}, \\ \text{SecurityParameters.server_random} \parallel \\ \text{SecurityParameters.client_random})$$

直至产生足够的输出为止。

握手协议-密码计算

- ❑ 密码参数产生。密码规格要求客户端/服务器写MAC值的密钥，客户端/服务器写密钥，客户端/服务器写初始向量IV，这些都是按顺序由主密钥产生的。其方法是主密钥利用散列函数来产生安全字节序列，字节序列足够长以便生成所有需要的参数。

- ❑ 从主密钥中计算密钥材料的方法如下：

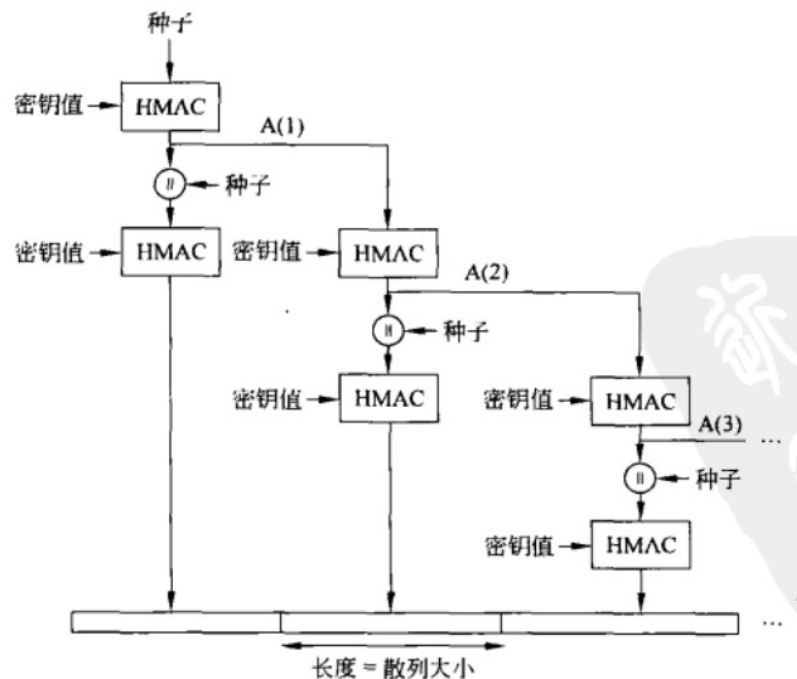
```
key_block=MD5 (master_secret || SHA('A' || master_secret ||  
                ServerHello.random || ClientHello.random)) ||  
            MD5 (master_secret || SHA('BB' || master_secret ||  
                ServerHello.random || ClientHello.random)) ||  
            MD5(master_secret || SHA('CCC' || master_secret ||  
                ServerHello.random || ClientHello.random)) || ...
```

- ❑ 该计算过程一直持续到产生足够长的输出，该算法结构的结果相当于一个伪随机函数。主密钥可以认为是伪随机函数的种子值。客户端和服务器的随机数可以认为是增加密码分析复杂度的“加盐”。



握手协议-密码计算

- ❑ TLS使用一个称为PRF的伪随机函数来扩展密钥以得到密钥产生和验证中的各种密钥块。
- ❑ 采用伪随机函数的目的是使用相对较小的共享密钥值，生成较长的数据块，防止对散列函数和MAC的攻击。PRF基于下面的数据扩展函数。



- ❑
$$\text{P_hash}(\text{secret}, \text{seed}) = \text{HMAC_hash}(\text{secret}, A(1) \parallel \text{seed}) \parallel$$
$$\text{HMAC_hash}(\text{secret}, A(2) \parallel \text{seed}) \parallel$$
$$\text{HMAC_hash}(\text{secret}, A(3) \parallel \text{seed}) \parallel \dots$$
- ❑ 其中，A()定义如下: $A(0) = \text{seed}$; $A(i) = \text{HMAC_hash}(\text{secret}, A(i-1))$

握手协议-密码计算

- ❑ 数据扩展函数使用以MD5或SHA-1作为基本散列函数的HMAC算法。从上面的定义中可以看出，P_hash可以根据迭代的次数来产生所需的数据量。
- ❑ 例如，如果要使用P_SHA-1产生64字节的数据，则需要迭代4次，先产生80字节的数据，然后将最后16字节丢弃。而如果采用P_MD5，同样需要迭代4次，生成恰好64字节的数据。值得注意的是，每迭代一次将涉及两次HMAC计算，其中的每一次计算又涉及两次散列算法的计算。
- ❑ 为了使PRF做到尽可能安全，同一种情况下可以使用两个不同的散列函数。如果这两个函数中有一个是安全的，那么其安全性就可以得到保证。这种情况下PRF的定义如下：
$$\text{PRF}(\text{secret}, \text{label}, \text{seed}) = \text{P_MD5}(\text{S1}, \text{label} \parallel \text{seed})$$
- ❑ PRF包含三个输入，分别是密钥值、标识符和种子。通过这些参数可以产生一个任意长的输出。

心跳协议

- ❑ 在计算机网络环境中，心跳是由硬件或软件产生的周期性信号以指示正常操作或同步系统的其他部分。心跳协议通常用于监视协议实体的可用性。在TLS的特定情况下，2012年在TLS中定义了心跳协议。
- ❑ 心跳协议运行在TLS记录协议之上，并且由两种消息类型组成：心跳请求(heartbeat_request)和心跳响应(heartbeat_response)。心跳协议的使用是在握手协议的第一阶段中建立的。
- ❑ 一条心跳请求信息可以在任何时间发出。任何时候只要接收一条请求信息，都应该及时地应答一条相应的心跳响应信息来对其进行响应。
- ❑ 心跳请求信息包括负载长度、负载和填充字段。负载是长度在16字节到64千字节之间的随机内容。相应的心跳响应信息必须包括接收到的负载的精确复制。填充字段也是随机内容。
- ❑ 心跳协议的目的是，它可以向发送方保证接收方是存活的，即使底层TCP连接上已经一段时间没有活动了。其次，心跳协议在空闲时期会在连接中产生活动，以避免被不兼容空闲连接的防火墙关闭。

SSL/TLS攻击

- 自从1994年首次提出SSL，以及此后TLS标准提出以来，大量针对这些协议的攻击被设计出来。每种攻击的出现都迫使使用的协议、使用的加密工具或者SSL和TLS实施的一些方面做出改变，以应对这些威胁。
- BEAST攻击（Browser Exploit Against SSL/TLS）：这种攻击利用SSL3.0和TLS1.0中的密码块链（CBC）模式的实现漏洞，允许攻击者解密HTTPS会话中的信息。
- 攻击PKI：对一些攻击来说，检查SSL/TLS内容中或者其他地方的X.509证书的合法性都是一个很重要的主题。比如，SSL/TLS常用的库易受证书验证实施的影响。
- DOS攻击：通过使用SSL/TLS握手请求压倒目标，在服务器上造成了沉重的处理负载。通过建立新的连接或者使用重协商来增加系统的负载。假设在握手期间主要的计算是由服务器完成，这种攻击在服务器上产生的系统负载多于在源设备上产生的，就导致了DoS，服务器被迫不断重新计算随机数和密钥。

TLS v1.3

- 自从在2014年，IETF TLS工作组开始了TLS版本1.3的工作。最主要的目的是增强TLS的安全性。相对于版本1.2来说有以下一些显著的改变：
 - TLS v1.3取消了对一些操作和函数的支持。移除了执行不需要的函数代码以减少代码错误的潜在危险，并降低攻击面。
 - TLS v1.3使用Diffie-Hellman或者椭圆曲线Diffie-Hellman来作为密钥交换，并且不允许RSA。RSA的危险性在于如果密钥被泄露了，那么使用这个密码组的所有握手都会被泄露。使用DH或者ECDH，每次握手都会有一个新的密钥重协商。
 - TLS v1.3通过改变创建安全连接发送的消息顺序，可以实现“1往返时间”的握手。在一个密码组被重协商之前，客户端发送一条包含密钥创建加密参数的Client Key Exchange信息。这使得服务器在发送首次应答之前可以计算用于加密和认证的密钥。减少在握手阶段发送的数据包从而加速进程，并降低攻击面。

HTTPS

- ❑ HTTPS是指用HTTP和TLS的结合来实现网络浏览器和服务端之间的安全通信。HTTPS被加载到所有现代网络浏览器中。它的应用依赖于网络服务器支持的HTTPS通信。
- ❑ 在浏览器用户看来，它们的主要的区别表现在URL地址开始于https://而不是http://。一个标准的HTTP连接使用80端口，而HTTPS使用443端口。
- ❑ 当使用HTTPS时，通信的以下元素被加密：
 - 要求的文件的URL。
 - 文件的内容。
 - 浏览器表单的内容(由浏览器的使用者填写)。
 - 从浏览器发送到服务器和从服务器发送到浏览器的Cookie。
 - HTTP标题的内容。

HTTPS

- ❑ **连接发起。**用户在合适的端口向服务器发起一个连接，然后发送TLS的ClientHello，开始TLS信号交换。
- ❑ 当TLS信号交换完毕后，用户将发起第一次HTTP请求。所有HTTP数据都要以TLS应用数据的形式发送。
- ❑ **连接关闭。**一个HTTP用户或者服务器可以通过在HTTP记录中加入“`connection:close`”的字样来指示一个连接的关闭。这意味着该连接将会在该条记录传输之后关闭。
- ❑ 关闭一个HTTPS连接要求关闭TLS与其对应的远程终端之间的连接，这要求关闭潜在的TCP连接。在TLS层面，关闭一个连接的适当做法是两端都是用TLS警报通信协议发出一个“`close_notify`”警告。

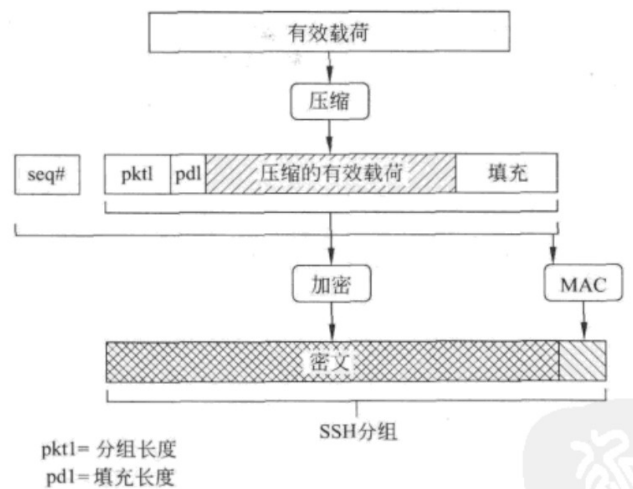
SSH

- ❑ SSH(Secure Shell)是一个网络信息安全通信协议，致力于提供一个安全的远程登录装置以代替其他远程的无保护的登录机制，支持文件传输和E-mail等网络功能。
- ❑ SSH由三个通信协议组织而成，通常运行于TCP之上。
 - 传输层协议:提供服务器身份验证、数据保密性、数据的完整性和未来保密。传输层会有选择地提供压缩。
 - 用户身份验证协议:验证服务器的用户。
 - 连接协议:在一个单一、基础的SSH连接上复用多个逻辑的通信渠道。

SSH 用户验证协议 服务器对客户端用户认证	SSH 连接协议 将加密隧道分拆到几个逻辑通道
SSH 传输层协议 服务器提供认证、机密性和完整性服务，也可能根据选择还会包括压缩	
TCP 传输控制协议。提供可靠性，按照连接进行端到端的投递	
IP 互联网协议通过多网络提供数据包投递	

SSH-传输层协议

- ❑ **分组交换。** 客户端向服务器建立一个TCP连接。当连接建立以后，用户端和服务器交换数据，被称为分组(packet)。每个分组采用如下形式：
 - **分组长度：**分组的字节长度，不包括分组的长度和MAC领域。
 - **填充长度：**随机的填充域的长度。
 - **有效载荷：**分组的有用内容。在计算程序协商之前，这个区域未被压缩。如果压缩算法协商之后，则在下一个分组这个区域就将被压缩。
 - **随机填充：**一旦一个加密算法被协商，这个域便被附上。它包括了填充的随机字节，因此分组的总长度(不算MAC区域)是密码块大小的一个倍数。
 - **消息身份验证码：**当协商好消息认证，这个域便有了MAC值。MAC值是由整个分组加上一个序列号计算得出的。



SSH-传输层协议

□ SSH传输层分组交换由以下步骤构成。

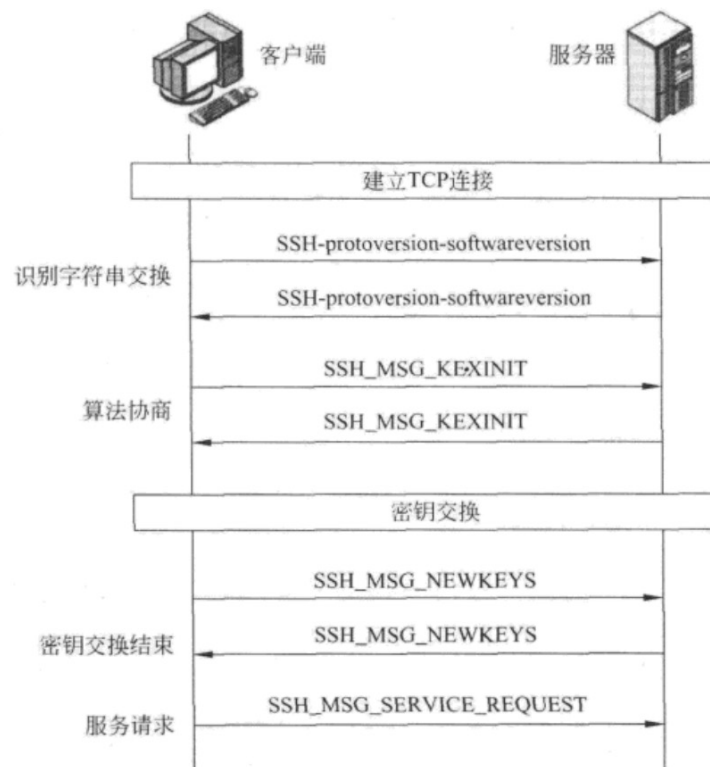
➤ 身份标识串交换，开始于客户端发送一个含有识别身份标识串的分组，该字符串的形式为：

SSH-protoversion-softwareversion SP comments CR LF

➤ 这里的SP、CR和LF分别是空格符、回车符和换行符。一个有效的字符串的例子为：

SSH-2.0-billsSSH_3.6.393<CR><LF>。这些字符串被用在Diffie-Hellman密钥交换。

□ 接下来是算法协商。每端各发出一个SSH_MSG_KEXINIT包含了一个支持算法的清单，算法包括密钥交换、加密、MAC算法和压缩算法。



SSH-传输层协议

密 码	
3des-cbc*	三密钥 3DES 的 CBC 模式
Blowfish-cbc	Blowfish 的 CBC 模式
twofish256-cbc	256 比特密钥 twofish 算法的 CBC 模式
twofish192-cbc	192 比特密钥 twofish 算法的 CBC 模式
twofish128-cbc	128 比特密钥 twofish 算法的 CBC 模式
aes256-cbc	256 比特密钥 AES 算法的 CBC 模式
aes192-cbc	192 比特密钥 AES 算法的 CBC 模式
aes128-cbc*	128 比特密钥 AES 算法的 CBC 模式
Serpent256-cbc	256 比特密钥 Serpent 算法的 CBC 模式
Serpent192-cbc	192 比特密钥 Serpent 算法的 CBC 模式
Serpent128-cbc	128 比特密钥 Serpent 算法的 CBC 模式
arcfour	128 比特密钥的 RC4 算法
cast128-cbc	CAST-128 算法的 CBC 模式

*=必需的。

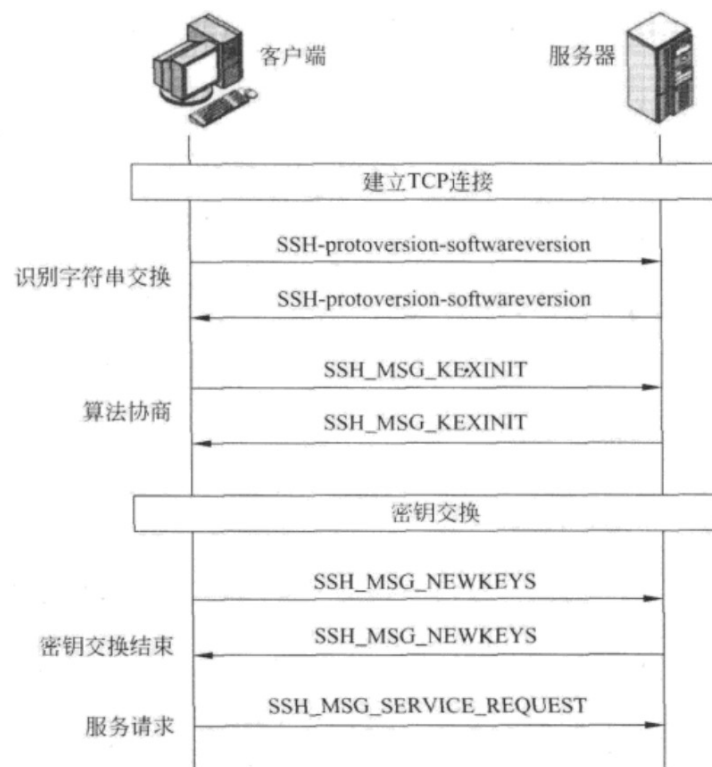
**=推荐的。

MAC 算法	
hmac-shal*	HMAC-SHA-1; 摘要长度=密钥长度=20 字节
hmac-shal-96**	HMAC-SHA-1 的前 96 比特; 摘要长度=12 字节; 密钥长度=20 字节
hmac-md5	HMAC-MD5; 摘要长度=密钥长度=16 字节
hmac-md5-96	HMAC-MD5 的前 96 比特; 摘要长度=12 字节; 密钥长度=16 字节

压缩算法	
none*	空算法, 没有压缩
zlib	RFC 1950 和 RFC 1951 中定义的算法

SSH-传输层协议

- 下一步是**密钥交换**。在目前，只有Diffie-Hellman密钥交换被明确可用。密钥交换是以SSH_MSG_NEWKEYS数据包的交换作为结束信号。
- 最后一步是**服务请求**。用户发送一个SSH_MSG_SERVICE_REQUEST数据包以请求获得用户身份验证或者连接(数据传递)协议。接下来，所有的数据会被作为SSH传输层数据包的有效载荷来交换，并将受到加密码和MAC的保护。



SSH-用户身份认证协议

- 用户身份验证协议提供用户得到服务器身份验证的方式。
- 消息类型和格式。在用户身份验证协议中经常用到三种消息类型。来自客户的身份验证请求有如下格式：

byte SSH_MSG_USERAUTH_REQUEST(50)

string user name

string server name

string method name

...

- 在此格式中，user name是客户端的身份标志，server name是客户端要求访问的设备(一般是SSH链接协议)，method name是在此请求中用到的认证方式。第一个字节的十进制值为50，这一数值被解读为SSH_MSG_USERAUTH_REQUEST。

SSH-用户身份认证协议

- 如果服务器(1)拒绝认证请求或者(2)接受请求但要求附加一个或多个认证方式，服务器会发送以下格式的信息：

byte	SSH_MSG_USERAUTH_FAILURE (51)
name-list	authentications that can continue
boolean	partial success
- 如果服务器接受身份认证，它会发送一个简单的字节消息:SSH_MSG_USERAUTH_SUCCESS (52)。

SSH-用户身份认证协议

□ 消息交换涉及以下步骤：

- (1) 用户不用通过任何被要求的方式，发送一个SSH_MSG_USERAUTH_REQUEST。
- (2) 服务器检查以决定用户名是否有效，如果无效，服务器发回SSH_MSG_USERAUTH_FAILURE，和请求失败的部分成功值。如果用户名有效，服务器进行第(3)步。
- (3) 服务器返回SSH_MSG_USERAUTH_FAILURE以及可被使用的一种或多种认证方式名单。
- (4) 用户选择一种被接受的身份认证方式，并将方式名称以及方式特定区域与SSH_MSG_USERAUTH_REQUEST一并发送出。在这点上，可能会有一系列的转换来执行此方式。
- (5) 如果用户身份认证成功而且被要求应用更多身份验证方式，服务器将进行第(3)步，显示的部分成功值为正确。如果身份认证失败，服务器将进行第(3)步，显示的部分成功值为错误。
- (6) 当所有被要求的身份认证方式都成功了，服务器会发送一条SSH_MSG_USERAUTH_SUCCESS信息，而身份验证协议结束。



SSH-连接协议

- SSH连接协议在SSH传输层协议之上运行，并认为一个安全身份认证连接正在使用中。连接协议使用安全身份验证连接(称为隧道)多路传输一定数量的逻辑信道。
- 信道机制。所有使用SSH的通信类型，例如一个终端会话，都由不同的信道支持。双方中的任意一方都可能开放一个信道。对于每个信道，每一方都有一个独一无二的序列号，两方的序列号不用一致。
- 信道是通过一个窗口机制进行流量控制的。只有当接收的信息指明有窗口空间时，才可能有数据发送到信道。

SSH-连接协议

- 一条信道的生命周期经历三个阶段：开放信道、数据传送和关闭信道。
- 当双方中的任一方想要开放新信道时，它将给信道分配一个本地号码，并以如下格式发送信息：

byte SH_MSG_CHANNEL_OPEN

string channel type

uint32 sender channel

uint32 initial windows size

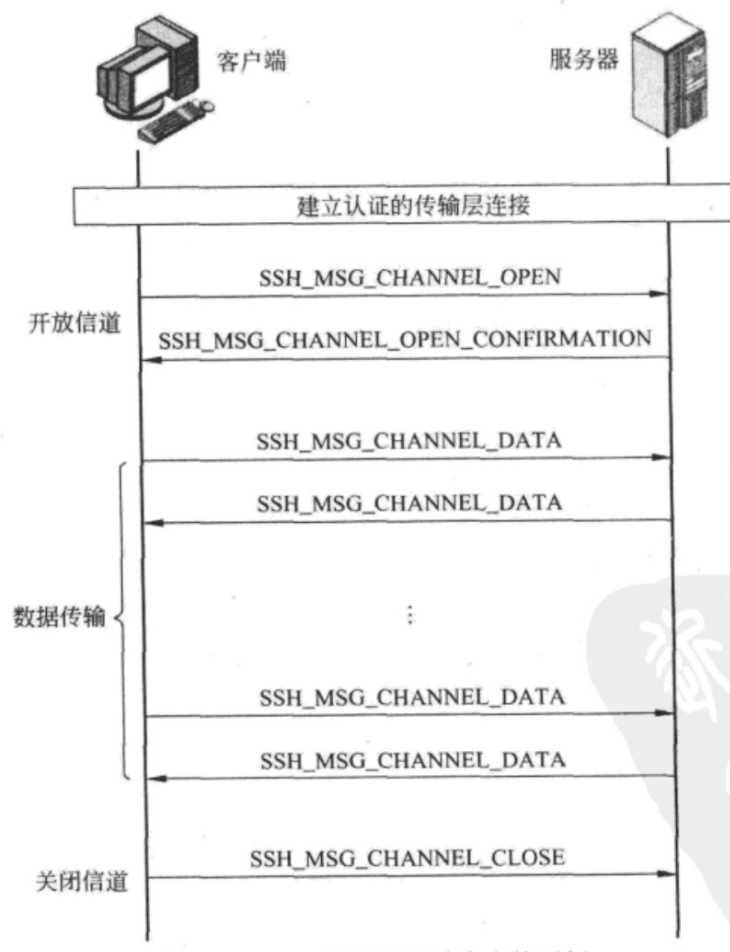
uint32 maximum packet size

...

- channel type标志了该信道使用的应用程序。sender channel是本地信道号码。initial windows size是指在不调整窗口的情况下，能向发送者传输信息中多少字节的信道数据。maximum packet size是指能向发送者发送的独立数据包最大尺寸。

SSH-连接协议

- 如果远程一方能够开启信道，它将回复SSH_MSG_CHANNEL_OPEN_CONFIRMATION信息，这条信息包括发送者信道号码、接收者信道号码，以及窗口和数据包对于即将发生的传输给出的尺寸数值。否则，远程的一方会回复一条SSH_MSG_CHANNEL_OPEN_FAILURE信息，和一条失败原因的报错码。
- 一旦一条信道被开放，数据传送通过SSH_MSG_CHANNEL_DATA信息执行，这条信息包括接收者信道号码以及一组数据。只要这条信道是开放的，这些信息可能进行不断的双向传输。
- 当双方中的任一方想要关闭信道时，它会发送SSH_MSG_CHANNEL_CLOSE信息，这条信息包括接收者信道号码。

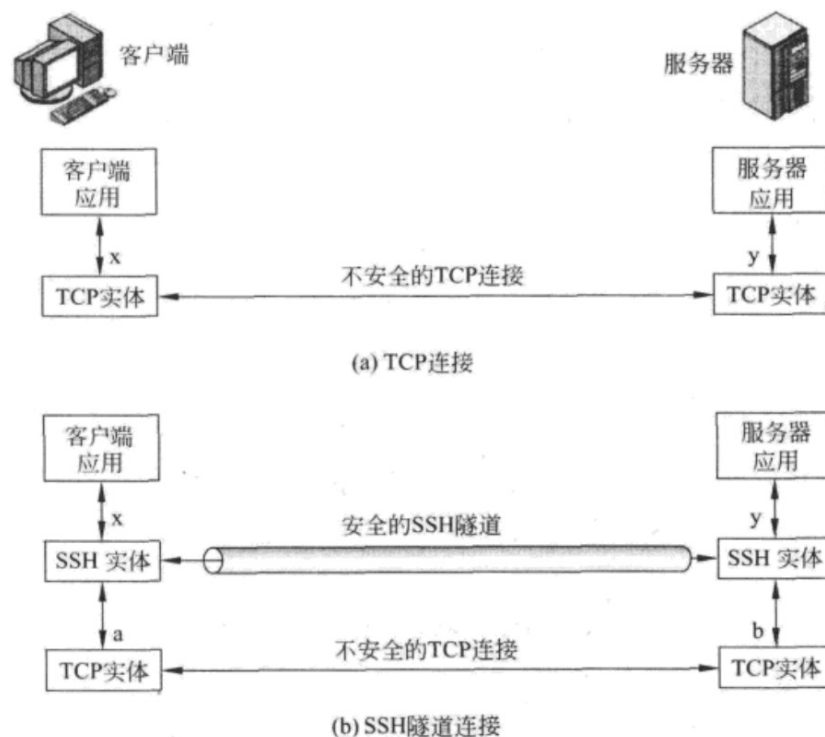


SSH-连接协议

- 信道类型。在SSH连接协议细则认证四种信道类型：
 - 对话：一个程序的远程执行。这个程序可能是Shell，或者电子邮件，一个系统指令，或者一些内在的分系统。一旦一个对话信道被打开，连续的请求将被用于开启远程程序。
 - X11：这是指X窗口系统，一种为联网计算机提供图形用户界面的计算机软件系统和网络协议。X允许应用程序运行在网络服务器，但会显示在台式机上。
 - 向前tcpip：远端端口转发。
 - 直接tcpip：这是本地端口转发。
- 端口转发：SSH最有用的特征之一就是端口转发。本质上来说，端口转发能够将任何不安全的TCP连接转换成安全的SSH连接，这也称为SSH隧道技术。
- 一个端口是一个TCP用户的标识符。所以，任何运行在TCP上的应用程序都具有一个端口号。基于该端口号，TCP链路向合适的应用程序发送数据。

SSH-连接协议

- 在端口转发中，我们有一个被端口数 x 所识别的客户端应用程序和一个被端口数 y 识别的服务器应用程序。有些情况下，客户端应用程序调用本地TCP实体并需要一个连接到达在端口 y 上的远端服务器。
- 为了确保这个连接的安全，SSH被配置以便SSH传输层协议分别在SSH客户端和带有TCP端口号 a 和 b 的服务器实体间建立一个TCP连接。一个安全的SSH通道被建立在TCP连接上。从端口 x 上的客户端来的传输，被重新传入到本地SSH实体，传输通过通道，那里的远端SSH实体传递数据到端口 y 上的服务器应用程序。在其他方向上的传输也被相似的重新传入。



SSH-连接协议

- ❑ SSH提供两种类型的端口转发：本地转发和远端转发。
- ❑ **本地转发**允许客户端建立一个“hijacker”的进程。这将拦截选择的应用层传输并将其从一个不安全的TCP连接重新传入到安全的SSH通道中。SSH被配置可以使用所选择的端口。SSH利用一个所选择端口抓取所有传输，并通过一个SSH通道发送它。在另一个末端，SSH服务器被客户端应用程序命令发送输入传输到终点端口。
- ❑ 举例：假设你有一个邮件客户端，并利用它从你的邮件服务器经由接收邮件服务器(POP)获取你的电子邮件。POP3被分配的端口数是端口110。我们可以通过以下几步来确保这个传输：
 - SSH客户端建立一个连接到远程服务器。
 - 选择一个没有使用的本地端口数，比如9999，然后配置SSH接收传输从这个端口到服务器上的端口110。
 - SSH客户端通知SSH服务器创建一个连接到终端，这时邮件服务器端口为110。
 - 客户端将所有的字节发送到本地端口9999，并将他们送到服务器内加密SSH会话。SSH服务器解密输入字节然后将明文传至端口110。
 - 另一个方向，SSH服务器获取在端口110上接收的所有字节，并将他们送入SSH会话后返回客户端，客户端解密数据，然后将他们送入端口9999的进程。

SSH-连接协议

- ❑ 在**远端转发**，用户的SSH客户端替代服务器行动。客户端利用一个已给的终端端口数接收传输，将数据包放在正确的端口上并送它到用户指定的终端。
- ❑ 举例：工作的时候你希望从家用计算机进入一个服务器。因为工作服务器在防火墙的后面，所以它不会接受从你的家用计算机发来的SSH请求。但是，在工作中你可以利用远程转发建立一个SSH通道。通过以下几个步骤：
 - 从工作的计算机建立一个SSH连接到你的家用计算机。防火墙允许这些，因为这是一个出连接保护。
 - 配置SSH服务器使用本地端口，比如22，然后传输数据通过SSH连接到远端端口，比如2222。
 - 现在你可以到你的家用计算机旁，配置SSH在端口2222上接受传输。
 - 现在你有了一个SSH通道，可以被利用于远端登录到工作服务器上。

IPSec

以TCP/IP协议簇为基础的当代互联网，具有结构简单、易于实现的特点。TCP/IP也成为互联网事实上的国际标准。互联网安全的很大程度上取决于TCP/IP的安全能力。

在因特网中，存在许多为保障不同层次安全需求的特制安全协议。如：

□ HTTPS、SET

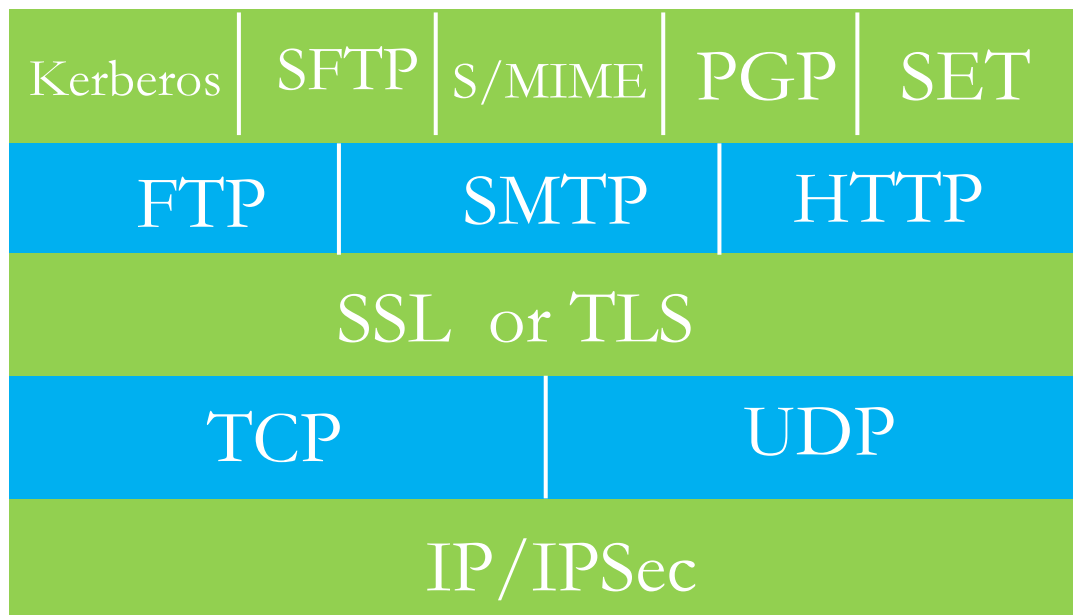
□ SSL/TLS

□ IPSec

应用层

传输层

网际层



IPSec

1998年，针对TCP/IP的先天缺陷，互联网工程任务组（Internet Engineering Task Force，IETF）发布了网际层的安全标准IPSec（IP Security）协议，并于2005年发布了更加完善的版本。

IPSec的目标是为IP网络（IPv4和IPv6）提供具有较强的互操作能力、高质量和基于密码的安全，在网际层实现多种安全服务，包括：

- ❑ 访问控制；
- ❑ 无连接完整性；
- ❑ 数据源认证；
- ❑ 抗重放；
- ❑ 基于加密的机密性；
- ❑ 业务量机密性（有限）

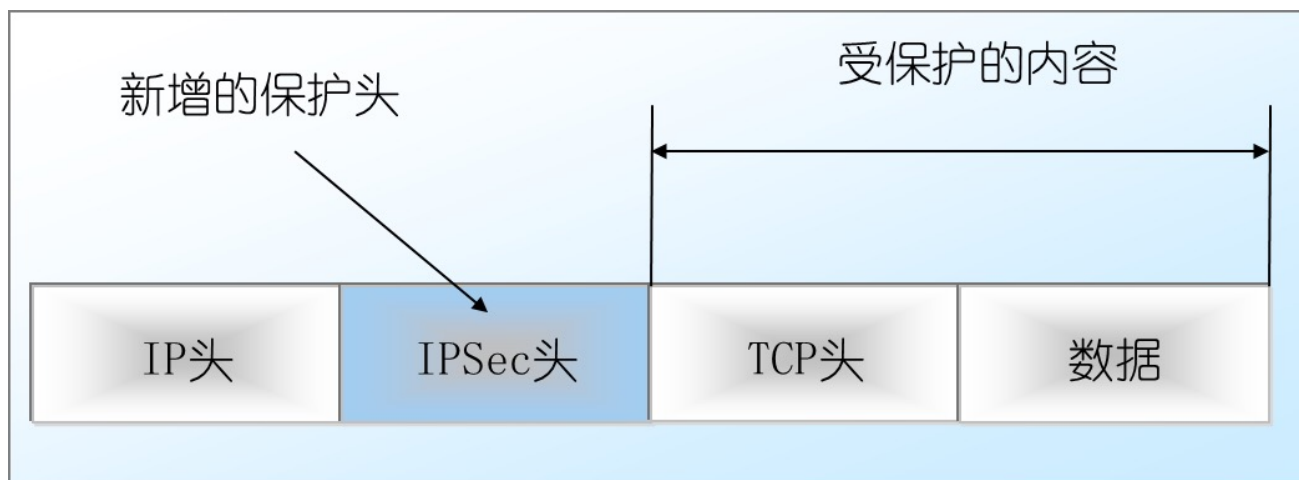
IPSec

IPSec的两种传输模式

IPSec中定义了两个重要的运行模式：传输模式和隧道模式。IPSec的安全服务能力均以此为基础。

(1) 传输模式

在IP头与上层协议头之间插入一个特殊的IPSec头。传输模式保护的是IP包的有效载荷（也即是TCP、UDP和ICMP等的报文）。通常，传输模式只用于两台主机之间的安全通信。



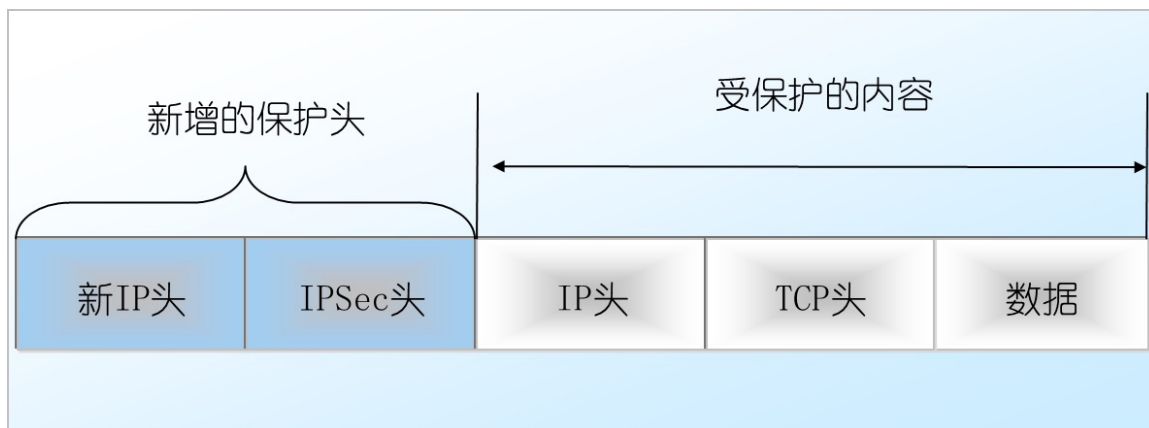
IPSec

(2) 隧道模式

隧道（**Tunneling**）是将需要保护的IP包被完整的封装到另一个IP包中，并在两个IP包头之间加上一个IPSec头。新的IP包的地址通常是防火墙、路由器或安全网关等。

当数据包通过这个隧道从IP网的一端传递到另一端时，沿途的路由器都只检查最外面的IP报头，不检查内部原来的IP报头，因而可以隐藏真实的地址。

对IPSec而言，IP隧道的直接目的就是对整个IP包提供完全的保护。



IPSec

2. IPSec的两个重要协议

IPSec中有两个主要的安全协议：**AH**和**ESP**。IPSec的安全服务均这两个协议提供。

(1) AH协议

验证头协议（**Authentication Header, AH**）是IPSec中最主要的协议之一，用于增强IP层安全，可以提供无连接的数据完整性、数据来源验证和抗重放攻击服务。

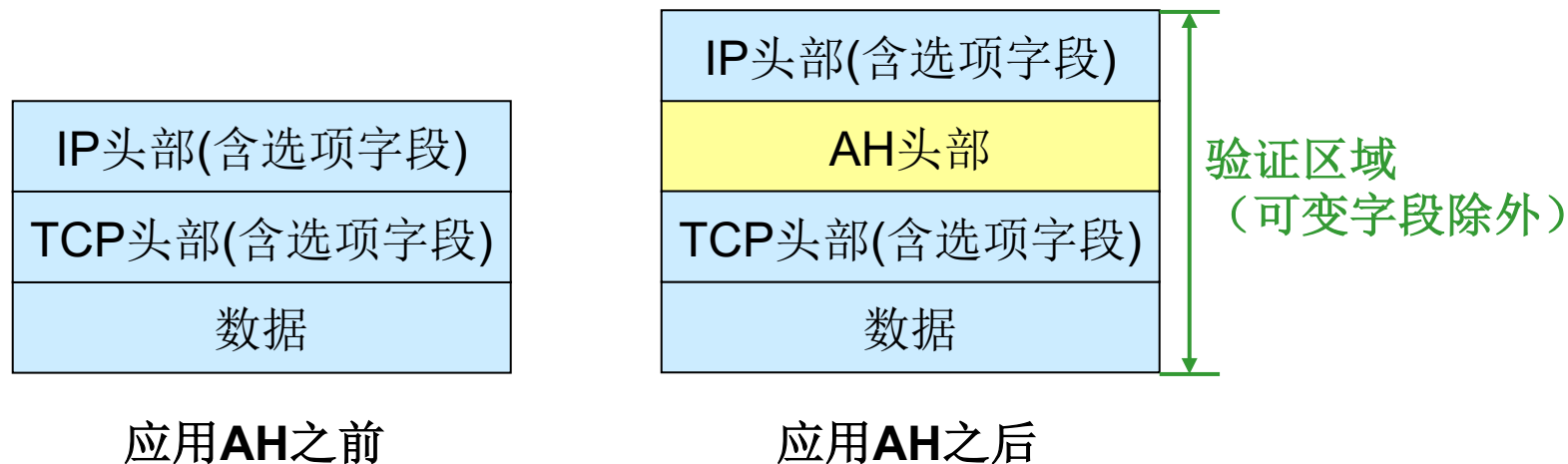
0	7	15	31
下一个报头	载荷长度	保留	
安全参数索引（SPI）			
序列号			
验证数据（变长）			

IPSec

AH的传输模式

AH插入到IP头部（包括IP选项字段）之后，传输层协议（如TCP、UDP）或者其他IPSec协议之前。

传输模式中AH协议首部的插入位置：



AH验证的区域是除可变字段以外的整个IP包，包括IP包头部，因此源IP地址、目的IP地址是不能修改的，否则会被检测出来。

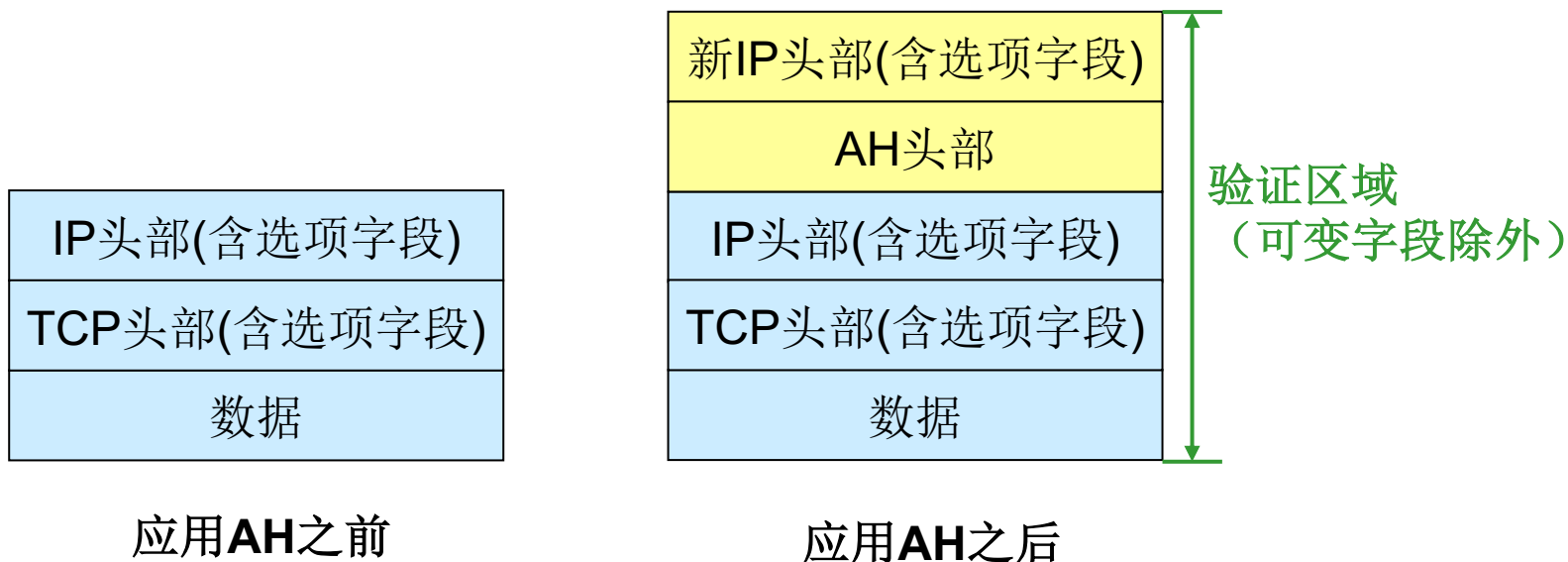
因此，意味着AH的传输模式不能支持NAT（Network Address Translation，网络地址转换）。

IPSec

AH的隧道模式

在隧道模式中，**AH**插入到原始IP头部之前，并在**AH**之前再增加一个新的IP头部。

隧道模式中AH协议首部的插入位置：



隧道模式下，**AH**验证的范围也是整个IP包，仍然存在**AH**与**NAT**的冲突。

IPSec

(2) ESP协议

封装安全载荷（Encapsulating Security Payload, ESP）：和AH协议一样，ESP协议也是IPSec中最重要的协议，并且得到了比AH更为广泛的应用。除了可以提供无连接的完整性验证、数据来源验证和抗重放攻击服务之外，ESP还提供数据包加密和数据流加密服务。

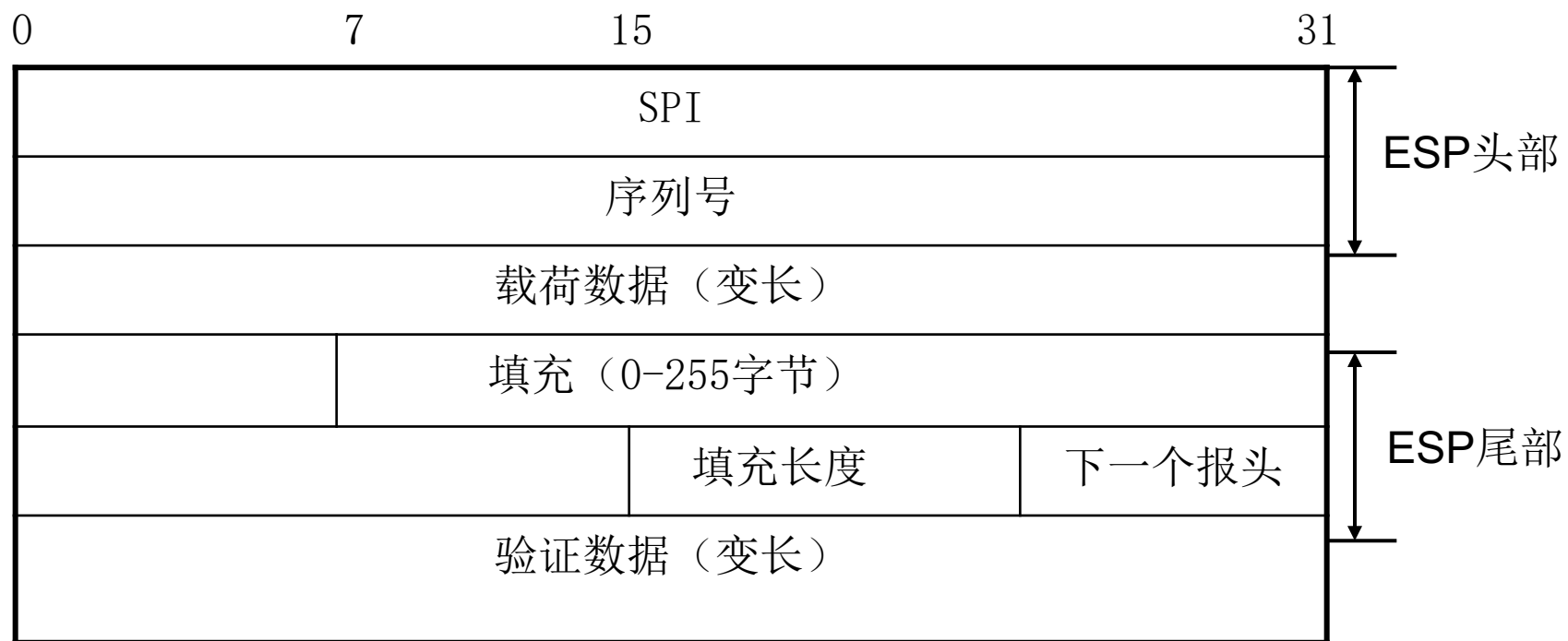
ESP的验证范围不包括IP包首部。ESP协议规定了所有IPSec系统必须实现的验证算法：HMAC-MD5、HMAC-SHA1、NULL。

ESP的加密采用的是对称密钥加密算法。不同的IPSec可以选择不同的加密算法。为了保证互操作性，ESP协议规定了所有IPSec系统都必须实现的加密算法：DES-CBC、NULL。

另外，ESP协议规定验证和加密不能同时使用NULL。

IPSec

ESP协议首部格式:

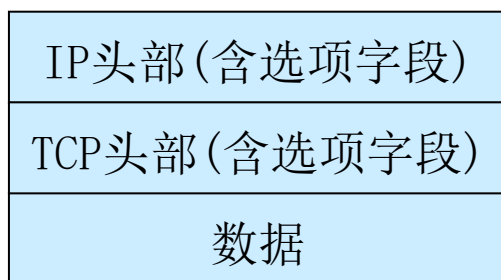


IPSec

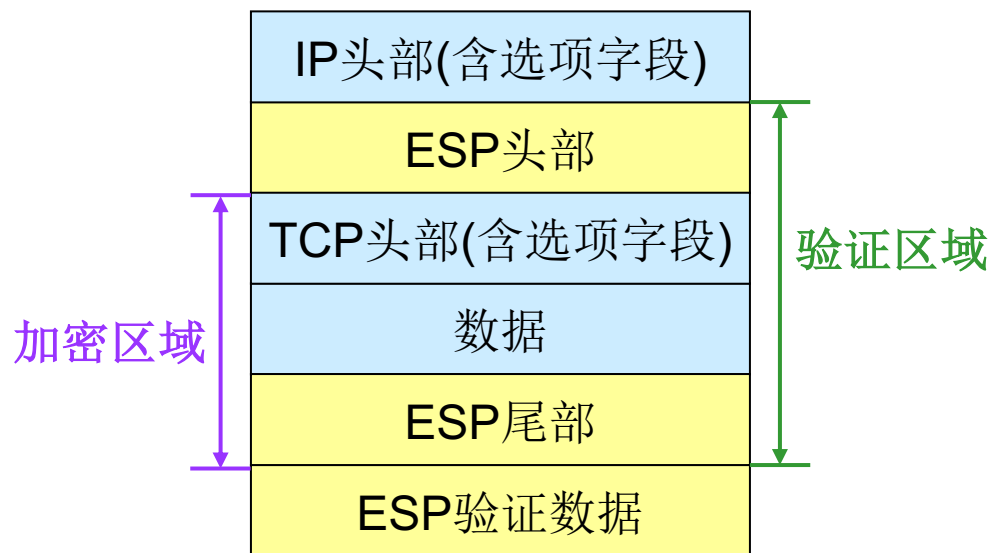
ESP的传输模式

在传输模式中，**ESP**协议保护的是**IP**包的载荷；**ESP**包头位于**IP**包首部（包括可选项）与**IP**载荷之间（如传输层协议**TCP**、**UDP**、**ICMP**，或者**IPSec**协议）。

传输模式中ESP协议首部与尾部的位置：



应用ESP之前



应用**ESP**之后

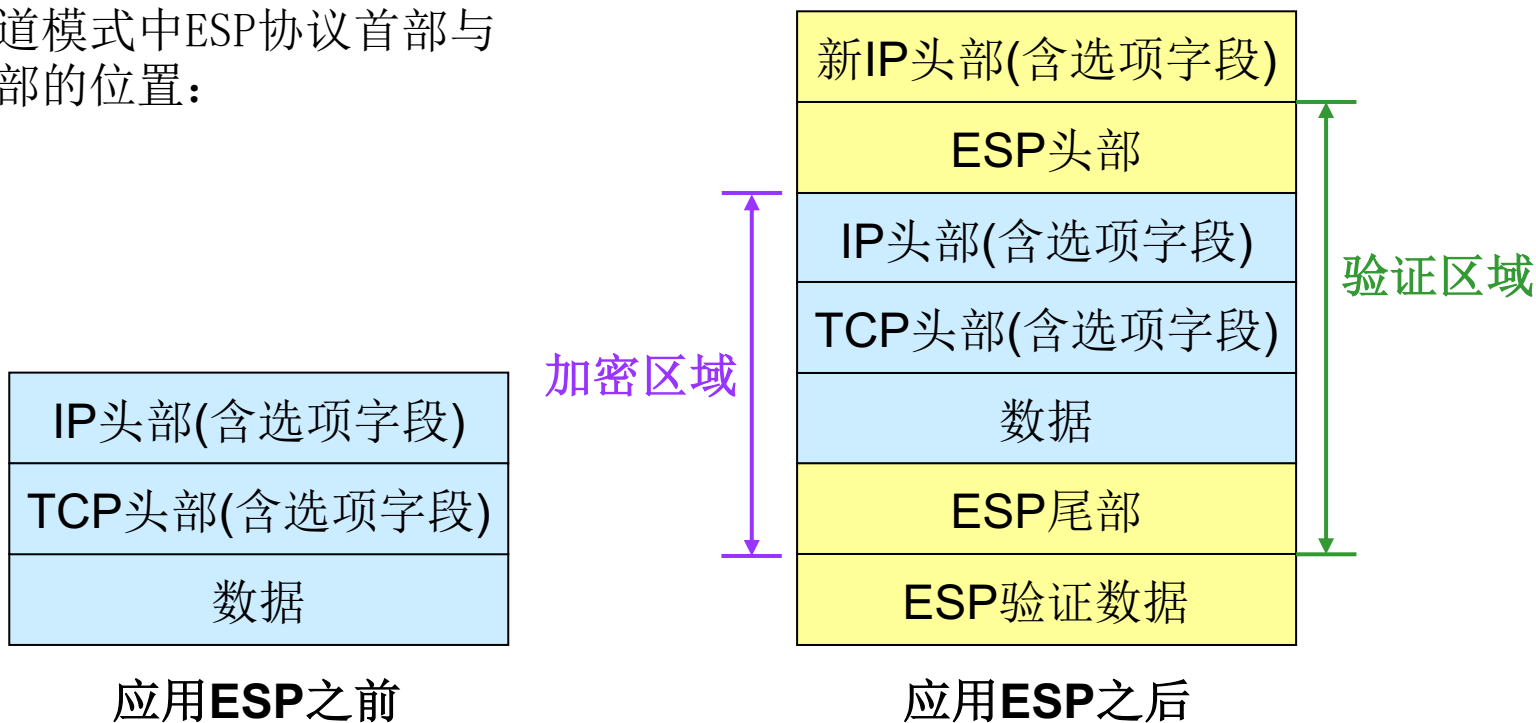
ESP验证的区域不包括**IP**包首部，因此可以在运行过程中修改**IP**包首部的内容，即不会和**NAT**发生冲突，但也无法检测出被修改。

IPSec

ESP的隧道模式

在隧道模式中，**ESP**保护的是整个**IP**包，对整个**IP**包进行加密；**ESP**包头插入到原始**IP**头部之前，并在**ESP**包头之前再增加一个新的**IP**头部。

隧道模式中ESP协议首部与尾部的位置：



隧道模式更加强大的安全功能，因为对整个原始**IP**包进行验证和加密，可以提供数据流加密服务，但是增加了一个额外的**IP**首部，将占用更多的带宽。与**NAT**不冲突。

IPSec

3、IPSec的安全联盟

安全关联（**Security Association, SA**）是两个IPSec实体（主机、安全网关）之间经过协商建立起来的一种协定，定义了一组与给定实体之间安全相关参数的集合，从而决定了保护对象、保护方式以及保护机制。

SA是一种单工关系，即业务流的发送方到接收方是一个单向逻辑关系。在双向的点到点通信中，需要两个**SA**，并构成一个**SA束（SA Bundle）**。

（1）SA的表示方法

每个**SA**由三元组（**SPI, IP目的地址, IPSec协议**）来惟一标识：

- ❑ 安全参数索引（**Security Parameter Index, SPI**），32位，标识具有相同IP地址和相同安全协议的不同的SA，被放在AH或ESP头中；
- ❑ IP目的地址：IP目的地址，是SA的终端地址；
- ❑ IPSec协议：采用AH或ESP。

IPSec

(2) 安全关联数据库

安全关联数据库（**Security Association Database, SAD**）并不是通常意义上的数据库，而是将所有**SA**以某种数据结构集中存储的一个表。

AH的安全关联数据库主要结构：

SPI	目标IP	执行协议	AH验证算法	AH加密密钥	工作模式
SPI值	IP地址	AH	HMAC-SHA1	#####	传输模式/ 隧道模式

ESP的安全关联数据库主要结构：

SPI	目标IP	执行协议	ESP验证算法	ESP验证算法加密密钥	ESP加密算法	ESP加密密钥	工作模式
SPI值	IP地址	ESP	HMAC-SHA1	#####	3DES-CBC	xxxxxxx	传输模式/ 隧道模式

IPSec + TLS

隧道模式 IPSec 与内部应用 TLS 结合，是典型的双层安全架构。外层 IPSec 对整个数据包加密封装，建立安全 VPN 隧道，保护网络层通信；内层 TLS 对应用数据端到端加密，保证业务安全。

外层：隧道模式 IPSec（网络层 VPN）

- 对整个原始 IP 包加密 + 认证
- 建立安全隧道；隐藏内部真实 IP、内部网络结构；保证数据包在公网上不被窃听、篡改、伪造
- 保护范围为整条网络链路。不关心上层是什么应用

内层：应用层 TLS（如 HTTPS）

- 对业务数据再加密
- 保护范围为应用到应用